

BI project

Instagram Engagement Analytics

Submitted By:

Rihem Abdelmoumen

Major: BA Minor: IT

September, 2026

TABLE OF CONTENTS

1. Introduction.....	2
2. Executive Summary.....	2
3. Dataset and Analytical Scope.....	3
4. Problem Statement and Objectives.....	7
5. Research Questions.....	10
6. ETL Process.....	11
6.1 ETL Overview.....	11
6.2 Pipeline Architecture.....	12
6.3 Extraction Stage.....	13
6.4 Transformation Stage.....	17
6.5 Configuration Management.....	41
6.6 Loading Stage.....	42
6.7 ETL Controls and Reproducibility.....	59
7. Data Warehouse Design.....	60
8. Power BI Dashboards.....	61
8.1 Overview.....	60
8.2 Category & Channel Breakdown.....	65
8.3 Performance Details.....	67
8.4 Outlier Impact and Distribution	68
8.5 Further Comparison.....	72
9. Key Findings.....	76
10. Business Recommendations	80
11. Analytical Limitations.....	85
12. Conclusion	87

1. Introduction

Instagram performance data can provide valuable evidence about audience interaction, content distribution, exposure, follower acquisition, and changes in performance over time. However, headline metrics may be misleading if they are interpreted without considering their definitions, exposure levels, distributional characteristics, and the potential influence of unusually large observations.

This project develops an end-to-end Business Intelligence solution for the descriptive analysis of Instagram post performance. The workflow integrates data extraction, data-quality treatment, feature and KPI engineering, dimensional modelling, relational storage, and interactive Power BI reporting. The analysis examines engagement, reach, impressions, follower growth, content characteristics, media formats, traffic sources, and time-based patterns.

A central distinction is made between **interaction volume** and **engagement efficiency**. Total engagement measures the absolute number of recorded interactions whereas engagement rate represents relative performance according to the exposure-based definition supplied in the source dataset. These measures answer different business questions and should be interpreted together rather than treated as interchangeable performance indicators.

The project also evaluates the sensitivity of engagement-rate results to extreme observations. An Interquartile Range (IQR) -based outlier flag is created during the transformation stage and retained in the fact table. This design enables Power BI users to compare full-data, non-outlier populations without permanently deleting potentially legitimate high-performing posts.

The results are descriptive and apply only to the observed dataset. The analysis identifies patterns, differences, associations, and robustness issues; it does not establish that a specific content category, media format, traffic source, or exposure level, or posting characteristic causes improved performance.

2. Executive Summary

The project delivers an integrated Business Intelligence solution for analyzing Instagram post performance across content categories, media formats, traffic sources, and reporting periods.

The solution combines:

- **Python** for data validation, cleaning, transformation, feature engineering, and outlier detection.

- **MySQL** for structured relational storage and star-schema-oriented dimensional modelling.
- **Power BI** for interactive dashboards, KPI monitoring, comparative analysis, filtering, and post-level drill-through.

The workflow transforms raw post-level data into a structured analytical model that supports consistent reporting of total engagement, engagement rate, reach, impressions, follower growth, time-based performance, and outlier-adjusted comparisons.

A key design decision is to flag statistically unusual engagement-rate observations rather than removing them from the dataset. This preserves the complete analytical population while enabling transparent comparison between raw and typical performance after excluding flagged outliers. The outlier indicator is incorporated directly into the fact table and can be used in Power BI slicers, DAX measures, tables, and visual comparisons.

The dashboards support analysis at multiple levels:

- Portfolio-level KPIs, including total engagement, reach, impressions, and engagement-rate.
- Comparative analysis by content category, media type, and traffic source.
- Monthly total-engagement trends and month-over-month growth.
- Identification of top posts by total engagement.
- Identification and exploration of engagement-rate outliers.
- Comparison of all-post and non-outlier engagement metrics.
- Drill-through from aggregated dashboard views to individual post records.

The solution is intended to support evidence-based monitoring and exploratory decision support. It is designed to identify patterns, quantify changes, reveal metric sensitivity, and generate hypotheses for future testing. It should not be used to infer causal effects from descriptive comparisons alone.

3. Dataset and Analytical Scope

3.1 Data Source

The analysis uses the Instagram Analytics dataset obtained from Kaggle. The source file, [Instagram_Analytics.csv](#), contains post-level Instagram performance records describing content characteristics, traffic distribution, audience exposure, interactions, follower acquisition, and a source provided engagement-rate field.

The CSV file is the common input for the ETL pipeline, MySQL data model, and Power BI reporting layer.

Attribute	Description
Platform	Kaggle
Dataset name	Instagram Analytics
Source file	Instagram_Analytics.csv
Source URL	Instagram Analytics Dataset
Observation unit	Individual Instagram post
Number of observations	29,999 posts
Reporting period	19 November 2024 to 19 November 2025
Raw variables	15
Processed analytical fields	21
Media types	4
Traffic sources	6
Content categories	10
Engagement-rate outliers	Approximately 8.5-9% of posts

Because the dataset is externally sourced, it should be treated as an analytical sample rather than a validated production export from an Instagram Business or Creator account. The findings therefore describe the supplied records and should not be generalized to other accounts, industries, audience populations, or geographical markets without further validation.

Each row represents one Instagram post and provides the raw attributes used throughout the analysis. The dataset is preserved in a staging layer, cleaned and enriched through Python-based transformation, structured into dimensions and a fact table, loaded into MySQL, and then analyzed in Power BI.

3.2 Industry and Organizational Context

The project is situated in the social-media marketing and digital-analytics domain, where organizations must assess large volumes of posts across categories, formats, distribution sources, and reporting periods. A centralized Business Intelligence solution reduces dependence on manual inspection and provides consistent KPI definitions, repeatable calculations, and interactive performance monitoring.

Social-media metrics require contextual interpretation. A post may generate high total engagement because it reaches a large audience, while another may have a

high engagement rate because its interactions are large relative to a smaller exposure base. Similarly, high impressions may reflect repeated exposure rather than a larger unique audience. An effective BI solution must therefore distinguish between scale, exposure, and efficiency.

The solution is designed for a hypothetical organization managing one or more Instagram content portfolios. It centralizes the post-level performance records and provides a common analytical environment for KPI reporting, exploratory analysis, outlier assessment, and post-level investigation.

The proposed solution supports:

- Standardized KPI definitions and reusable calculations.
- Comparison of content categories, media formats, and traffic sources.
- Monitoring of interaction volume, exposure, engagement rate, and follower growth.
- Analysis of monthly and period-based performance patterns.
- Identification and investigation of unusual engagement-rate observations.
- Comparison of full-data and outlier-filtered results.
- Drill-through from portfolio-level summaries to individual posts.

3.3 Observation Unit and Variables

The unit of analysis is an individual Instagram post. Each row represents one post and contains information about publication timing, media format, content category, traffic source, audience exposure, recorded interactions, follower growth, text-related characteristics, and engagement rate.

The raw dataset contains 15 variables organized into the following analytical domains.

Domain	Variables	Analytical role
Identification	<code>Post_id</code>	Identifies each post and supports record-level uniqueness checks
Time	<code>upload_date</code>	Supports daily, monthly, quarterly, and yearly analysis
Content format	<code>media_type</code>	Supports comparison of Photo, Video, Reel, and Carousel posts
Content classification	<code>content_category</code>	Supports category-level performance analysis
Traffic distribution	<code>traffic_source</code>	Identifies the recorded source associated with post traffic

Engagement	likes, comments, shares, saves	Measures recorded audience interactions
Exposure	reach, impressions	Measures audience exposure breadth and display volume
Audience growth	followers_gained	Measures followers gained in association with a post
Content characteristics	caption_length, hashtags_count	Describes selected text and publishing attributes
Rate KPI	engagement_rate	Source-provided engagement-rate value

3.4 Raw Dataset Variables

Variable	Type	Description
post_id	Identifier	Unique identifier assigned to an Instagram post
upload_date	Datetime	Date and time associated with post publication
media_type	Categorical	Media format used of the post
traffic_source	Categorical	Recorded source associated with post traffic
content_category	Categorical	Assigned content category
likes	Integer	Number of recorded likes
comments	Integer	Number of recorded comments
shares	Integer	Number of recorded shares
saves	Integer	Number of recorded saves
reach	Integer	Number of accounts recorded as reached
impressions	Integer	Total number of recorded content displays
followers_gained	Integer	Number of followers gained in association with the post
caption_length	Integer	Numerical measure of caption length
hashtags_count	Integer	Number of hashtags associated with the post.
engagement_rate	Decimal	Engagement-rate value provided in the source dataset

The dataset includes the following categorical values:

Variable	Value
<code>media_type</code>	Reel, Video, Photo, Carousel
<code>traffic_source</code>	Home Feed, Profile, Hashtags, External, Explore, Reels Feed
<code>content_category</code>	Technology, Photography, Fitness, Music, Beauty, Food, Lifestyle, Travel, Fashion, Comedy

3.5 Derived Analytical Fields

During ETL and analytical modelling, additional fields are created to support KPI calculation, time-based reporting, comparative analysis, and outlier-sensitive interpretation.

Derived Field	Purpose
<code>total_engagement</code>	Calculates total interaction volume as likes+comments+shares+saves
<code>year, quarter, month_number, month_name, year_month</code>	Supports time-based reporting and chronological analysis
<code>engagement_growth_rate</code>	Measures month-over-month change in total engagement
<code>high_engagement_flag</code>	Identifies posts exceeding the project-defined total-engagement threshold
<code>is_outlier</code>	Identifies engagement-rate values outside IQR-based bounds
<code>average_engagement_by_media</code>	Provides the average engagement reference value for each media type

The `is_outlier` field is deliberately retained in the fact table rather than used to delete records. This preserves the full population while enabling comparison of all posts, non-outliers, and outliers through Power BI slicers, DAX measures, tables, and visuals.

4. Problem Statement and Objectives

4.1 Problem Statement

Social media performance can be difficult to interpret when it is assessed through isolated KPIs. Likes, comments, shares, saves, reach, impressions, follower growth,

and engagement rate each provide useful but incomplete information. Individually, these metrics do not establish whether observed performance differences are stable, comparable across segments, adjusted for exposure, or influenced by exceptional observations.

Initial dashboards may identify apparent leaders across categories, media formats, traffic sources, or reporting periods. However, those rankings may change when a different KPI is selected or when unusual engagement-rate observations are excluded. For example, high total engagement may be driven by extensive exposure, while high average engagement rate may be materially affected by a small number of extreme values.

A robust Business Intelligence solution must therefore provide consistent KPI definitions, preserve data lineage, distinguish volume from efficiency, and test whether observed differences remain stable under alternative analytical conditions.

This project addresses the need for an integrated analytical framework that:

- Consolidates and standardizes post-level Instagram performance data.
- Supports reproducible KPI calculation and reporting.
- Distinguishes absolute interaction volume from engagement-rate performance.
- Compares results across content, traffic, and time dimensions.
- Assesses the influence of unusual engagement-rate observations.
- Enables exploration from portfolio-level summaries to post-level records.
- Produces decision-ready descriptive insights without overstating causal conclusions.

4.2 Business Problem

The primary business problem is the absence of a centralized and consistent framework for monitoring Instagram content performance across content categories, media formats, traffic sources, and reporting periods.

Without an integrated analytical solution, content and marketing teams may struggle to:

- Identify posts that generate the greatest interaction volume.
- Compare performance using consistent KPI definitions.
- Distinguish high exposure from high engagement efficiency.
- Assess media-type and traffic-source performance in context.
- Determine whether apparent performance leaders are influenced by unusual observations.
- Monitor time-based changes without overinterpreting short-term volatility.
- Reproduce analytical calculations across dashboards and reports.

- Convert high-volume post-level data into usable performance insights.

This project addresses these needs through a Python-to-MySQL ETL pipeline and a Power BI reporting layer. The resulting architecture supports analysis from portfolio-level metrics to individual post records.

The analysis is descriptive and observational. It identifies patterns and associations within the available data but does not establish that a category, media format, traffic source, content characteristic, or exposure measure causes improved performance. Causal evaluation would require additional variables and a stronger research design, including campaign strategy, paid-media activity, account-level characteristics, audience composition, publication practices, experimental controls, and treatment of confounding variables.

4.3 Analytical Objectives

4.3.1 Data Preparation and Modelling

- Validate, clean, and standardize the source data.
- Address missing values, invalid dates, inconsistent text labels, and duplicate records.
- Create derived fields required for KPI calculation, trend analysis, and outlier assessment.
- Load transformed data into a structured MySQL model.
- Develop a reusable data model for Power BI reporting.

4.3.2 Performance Measurement

- Measure total engagement, engagement rate, reach, impressions, and followers gained.
- Distinguish interaction volume from exposure-adjusted engagement performance.
- Assess portfolio-level, segment-level, and post-level results.
- Examine the distribution, variability, and concentration of key performance metrics.

4.3.3 Comparative Analysis

- Compare performance across content categories, media types, and traffic sources.
- Evaluate how results change when performance is measured using interaction, exposure, audience-growth, and engagement-rate metrics.
- Identify patterns that remain stable across measures.

- Interpret category, format, and source differences in the context of exposure and outlier effects.

4.3.4 Outlier and Robustness Analysis

- Identify unusual engagement-rate observations using the IQR method.
- Examine whether flagged observations are concentrated at particular exposure levels or within content dimensions.
- Compare full-data and non-outlier performance measures.
- Assess whether rankings and performance differences remain stable after excluding potentially influential observations.

4.3.5 Time-Based Analysis

- Examine monthly total engagement patterns.
- Calculate and evaluate month-over-month engagement growth.
- Identify periods of notable positive or negative change.
- Avoid unsupported claims of seasonality or causal drivers where the available time series is insufficient.

4.3.6 Dashboard Development

- Develop interactive Power BI dashboards for KPI monitoring and comparative analysis.
- Enable filtering by year, content category, traffic source, media type, and outlier status.
- Provide drill-through from aggregate results to individual post records.
- Implement DAX measures for all-post and non-outlier comparisons.
- Present metric definitions and relevant analytical limitations within the reporting environment.

4.3.7 Decision Support

- Generate reproducible and decision-relevant descriptive insights.
- Identify findings that are sensitive to metric selection, exposure differences, or outlier treatment.
- Support content-performance monitoring and hypothesis generation.
- Frame recommendations as testing opportunities rather than unsupported causal conclusions.

5. Research Questions

The investigation is guided by the following research questions:

1. What are the overall levels of interaction volume, exposure, follower acquisition, and engagement-rate performance in the analyzed Instagram portfolio?
2. How do total engagement, engagement rate, reach, impressions, and followers gained vary across the ten content categories?
3. How do content-category rankings change when performance is assessed using interaction-volume, exposure, follower-growth, and engagement-rate measures?
4. How do Reel, Video, Photo, and Carousel posts differ in total engagement, reach, impressions, followers gained, and engagement rate?
5. How does performance vary across Explore, Home Feed, Reels Feed, Profile, Hashtags, and External traffic sources when exposure and interaction metrics are evaluated together?
6. What descriptive relationships exist between reach, impressions, total engagement, engagement rate, and followers gained?
7. How does total engagement change over time, and what month-over-month engagement-growth patterns are observed during the reporting period?
8. Are unusual engagement-rate observations concentrated at particular levels of reach or impressions, or within particular content categories, media types, traffic sources, or reporting periods?
9. How do content-category, media-type, and traffic-source results change when engagement-rate outliers are excluded?
10. How does the distribution of engagement performance differ between full-data and outlier-filtered views?
11. Which observed patterns remain reasonably consistent across measures and time periods, and which are sensitive to extreme observations, exposure differences, or metric selection?
12. How can the ETL pipeline, MySQL data model, DAX measures, and Power BI dashboards support consistent content-performance monitoring while avoiding unsupported causal claims?

6. ETL Process

6.1 ETL Overview

The ETL pipeline converts the raw Instagram Analytics CSV file into a structured, validated, and analysis-ready data model for MySQL and Power BI. The process consists of three sequential stages:

1. **Extract:** Retrieve and preserve the raw source data.
2. **Transform:** Clean the data, derive analytical fields, detect outliers, and construct dimension and fact files.
3. **Load:** Create the MySQL schema and insert the processed files into the dimensional model in dependency order.

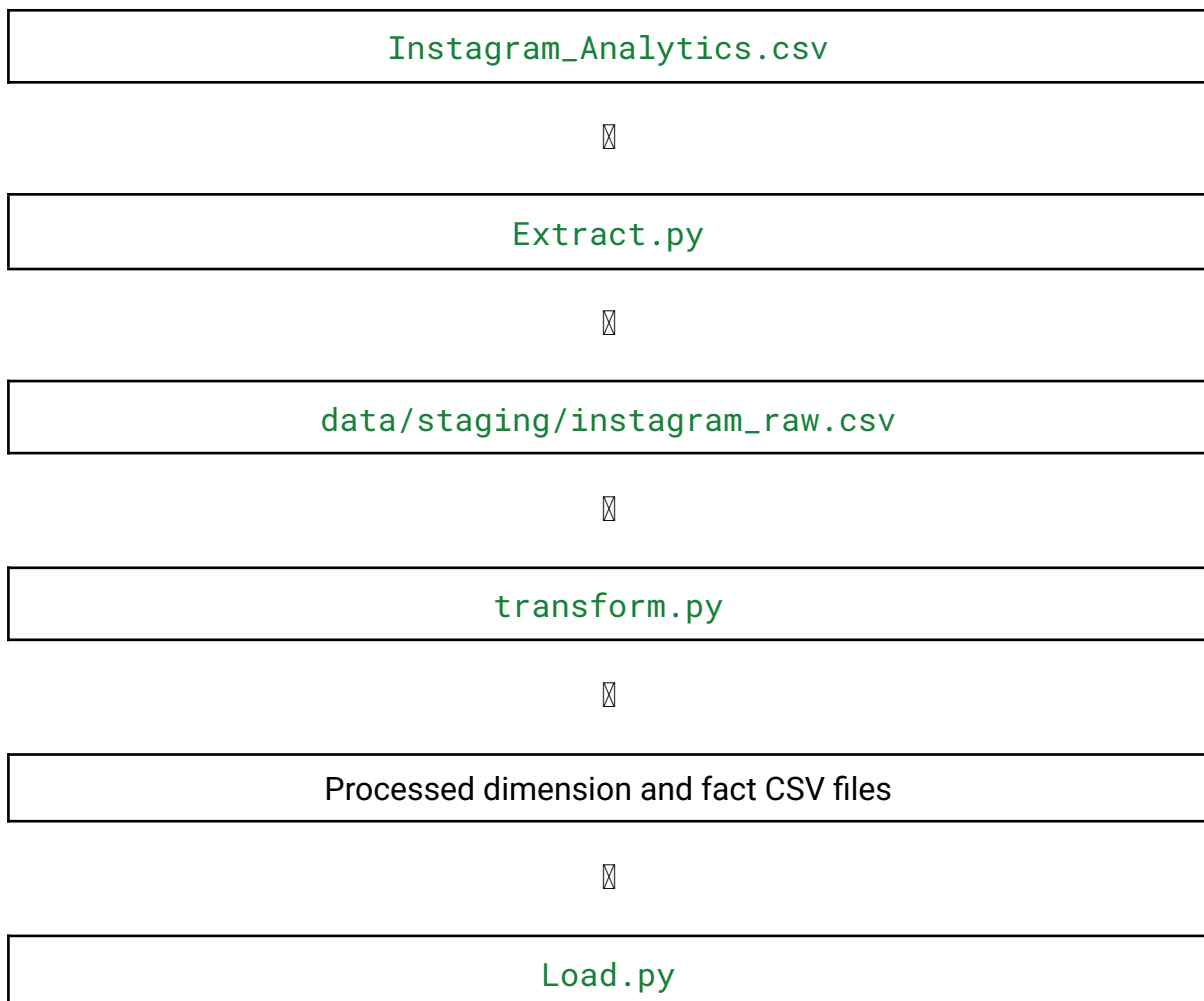
Stage	Script	Main responsibility
Extract	<code>Extract.py</code>	Reads the source CSV and saves a raw staging copy
Transform	<code>transform.py</code>	Cleans data, engineers KPIs, detects outliers, and builds analytical files
Load	<code>Load.py</code>	Creates the MySQL schema and loads dimension and fact tables

The pipeline preserves the original data, separates cleaning from analytical transformation, and validates the processed fact table before database loading.

This structure improves traceability, reproducibility, and consistency between the source dataset, analytical model, and dashboard layer.

6.2 Pipeline Architecture

The pipeline follows this flow:





MySQL warehouse



Power BI

The processed outputs include:

- `Instagram_Analytics_clean.csv`
- `Instagram_Analytics_clean_with_KPIs.csv`
- `time_dim.csv`
- `content_dim.csv`
- `media_dim.csv`
- `traffic_dim.csv`
- `instagram_fact.csv`
- `engagement_correlation_report.csv`
- `category_outlier_check.csv`

6.3 Extraction Stage

6.3.1 Purpose and Main Activities

The `Extract.py` script retrieves the source dataset, stores it in the staging area, and preserves a raw copy for downstream processing.

The script is intentionally limited to extraction activities. It does not clean, transform, aggregate, or modify the source records. This separation preserves data lineage and ensures that transformation results can be traced back to the extracted input.

The script performs the following steps:

1. Imports the required Python libraries.
2. Identifies the project root dynamically.
3. Defines and creates the staging directory.
4. Reads `Instagram_Analytics.csv` into a Pandas DataFrame.
5. Saves the extracted data as `instagram_raw.csv`.
6. Displays a completion message.

6.3.2 Imported Libraries

```
import pandas as pd
import os
```

Pandas provides the `DataFrame` structure used to represent the source data. It is used to:

- Read the CSV file into a DataFrame with `pd.read_csv()`.
- Export the extracted data with `to_csv()`.

The alias `pd` is the standard convention for Pandas.

The operating-system module (`os`) manages directories and file paths. It is used to:

- Locate the running script.
- Build portable directory and file paths.
- Create the staging directory.
- Convert relative paths into absolute paths.

Using `os.path` avoids dependence on a specific working directory and improves portability across development environments.

6.3.3 Project Root Resolution

```
project_root = os.path.abspath(os.path.join(os.path.dirname(__file__), ".."))
```

The project-root statement resolves the directory relative to the location of `Extract.py`.

Component	Purpose
<code>__file__</code>	Path of the currently executed script
<code>os.path.dirname(__file__)</code>	Directory containing <code>Extract.py</code>
<code>os.path.join(..., "..")</code>	Moves one level above the script directory
<code>os.path.abspath(...)</code>	Converts the result into an absolute path

Using `__file__` makes the script independent of the directory from which it is executed. It can therefore run from an IDE, terminal, scheduler, or other environment without requiring manual path changes.

6.3.4 Staging Directory Definition

```
staging_dir = os.path.join(project_root, "data", "staging")
```

The script constructs the path to the staging directory:

project_root/

└─ data/

 └─ staging/

The staging area acts as an intermediate storage layer between the source file and the transformation process. It separates:

- The original source file.
- The extracted raw copy.
- The transformed analytical files.
- The warehouse-ready data.

This separation supports traceability and makes it possible to reproduce the transformation process from a preserved input.

6.3.5 Staging Directory Creation

```
os.makedirs(staging_dir, exist_ok=True)
```

`os.makedirs()` creates the staging directory and any missing parent directories.

The parameter `exist_ok=True` prevents an error if the directory already exists.

This makes the extraction script repeatable: users do not need to create `data/staging` manually before each execution.

6.3.6 Source File Location and Reading

```
instagram_csv_path = os.path.join(staging_dir, "Instagram_Analytics.csv")  
instagram_data = pd.read_csv(instagram_csv_path)
```

The script constructs the path to `Instagram_Analytics.csv` and loads it into a DataFrame named `instagram_data` using `pd.read_csv()`.

At this stage, the dataset remains unchanged. No records are removed, standardized, aggregated, or enriched. The DataFrame serves as the in-memory representation of the source data before the raw copy is created.

6.3.7 Raw Extract Preservation

```
instagram_data.to_csv(os.path.join(staging_dir, "instagram_raw.csv"), index=False)
```

The extracted DataFrame is exported as `instagram_raw.csv` in the staging directory.

The parameter `index=False` prevents Pandas from writing its internal index as an additional column. The resulting file preserves the original tabular structure and provides a stable input for the transformation stage.

Maintaining this raw copy supports:

- Data lineage.
- Reproducibility.
- Comparison between source and transformed data.
- Recovery if later transformation logic changes.

6.3.8 Completion Message and Execution Feedback

```
print("Instagram data extraction completed and saved to staging area.")
```

The final instruction displays a message confirming that the script reached the end of the extraction workflow.

This confirms successful script execution but does not constitute formal data validation. Row counts, column types, missing values, and file contents are validated in later stages.

6.3.9 Extraction Workflow

The extraction script follows this sequence below:

Step	Description
Locate project root	Resolve the project directory relative to <code>Extract.py</code>
Define staging directory	Construct the path to <code>data/staging</code>
Create staging directory	Create the missing folders automatically

Read source CSV	Load <code>Instagram_Analytics.csv</code> into a DataFrame
Save raw copy	Export the data as <code>instagram_raw.csv</code>
Confirm Completion	Display an execution message

6.3.10 Extraction Result

```

● PS C:\Users\abdel> py C:\Users\abdel\Instagram_Project\etl\Extract.py
Instagram data extraction completed and saved to staging area.
○ PS C:\Users\abdel> 

```

Overall, `Extract.py` establishes a portable and repeatable extraction process. By resolving paths dynamically, creating the staging directory automatically, and preserving a raw data copy, it provides a controlled starting point for the transformation and loading stages.

6.4 Transformation Stage

6.4.1 Purpose and Main Activities

The `transform.py` script converts the staged dataset into cleaned, enriched, and analysis-ready outputs for MySQL and Power BI.

The transformation stage performs:

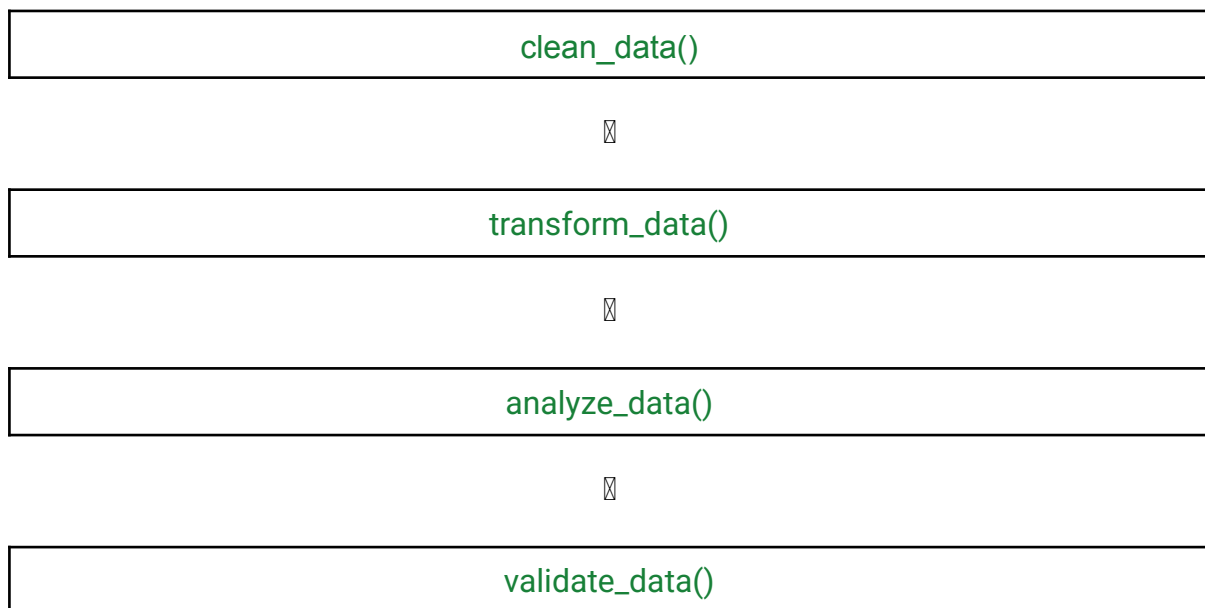
- Data cleaning and standardization.
- Missing-value treatment.
- Date validation.
- Duplicate removal.
- KPI and feature engineering.
- Monthly aggregation and growth-rate calculation.
- High-engagement classification.
- IQR-based outlier detection.
- Dimension-table construction.
- Fact-table construction.
- Exploratory category and correlation analysis.
- Final data-quality validation.

The result is a reproducible analytical model that supports both standard KPI reporting and outlier-sensitive analysis.

6.4.2 Transformation Workflow

Stage	Description
Setup	Sets the project directory and creates the processed-data folder
Data cleaning	Treats missing values, invalid dates, inconsistent labels, and duplicate posts
KPI engineering	Creates engagement, time, growth, classification, media, and outlier fields
Dimensional modeling	Builds time, content, media, and traffic tables
Fact table modeling	Builds the fact table
Exploratory analysis	Produces correlation and outlier-adjusted category reports
Validation	Checks for unexpected null values before loading

The `main()` function orchestrates the process in dependency order:



Cleaning is completed before KPI creation, analytical outputs are generated from the enriched data, and the fact table is validated before it is passed to the loading stage.

6.4.3 Imports and Environment Setup

```
import pandas as pd
import os
```

Pandas is the primary data-manipulation library used throughout the transformation process. It supports:

- Reading CSV files into DataFrames.
- Inspecting dimensions and missing values.
- Imputing missing data.
- Converting columns to datetime format.
- Standardizing categorical values.
- Removing duplicates.
- Creating calculated KPI fields.
- Grouping and aggregating records.
- Merging dimension identifiers.
- Exporting processed files.
- Performing exploratory analysis (correlation and outlier-adjusted category).

The **Operating-system module (os)** manages directories and working paths.

```
# ----- Setup -----
os.chdir(r"C:\Users\abdel\Instagram_Project")
print("Current working directory:", os.getcwd())

processed_dir = "data/processed"
os.makedirs(processed_dir, exist_ok=True)
```

`os.chdir()` changes Python's current working directory to the project root. Relative paths such as `data/staging/Instagram_Analytics.csv` and `data/processed/instagram_fact.csv` are then resolved from that location.

The `r` prefix creates a raw string, allowing Windows backslashes to be interpreted literally. `os.getcwd()` displays the active directory and provides a basic execution check.

The processed-data directory is created with:

```
os.makedirs(processed_dir, exist_ok=True)
```

This allows the script to be rerun without failing when the directory already exists.

6.4.4 Data Cleaning

The `clean_data()` function prepares the staged dataset for analytical modeling.

```
def clean_data():
```

It performs the following operations:

1. Loads the staged dataset.
2. Displays the initial dimensions.
3. Reviews missing values.
4. Imputes missing categorical fields with the mode.
5. Imputes missing numerical fields with the median.
6. Converts `upload_date` to datetime.
7. Removes records with invalid or missing dates.
8. Standardizes categorical text.
9. Removes duplicate `post_id` records.
10. Reviews missing values after treatment.
11. Saves the cleaned dataset.
12. Returns the cleaned DataFrame.

Loading and inspecting the staged data

```
df = pd.read_csv("data/staging/Instagram_Analytics.csv")
print("Initial dataset shape:", df.shape)
```

`pd.read_csv()` loads the staged file into a DataFrame named `df`.

The expression `df.shape` returns (number of rows, number of columns)

Displaying the initial shape creates a control point for comparing dataset volume before and after cleaning.

Missing-value assessment

The script first counts missing values by column:

```
print(df.isnull().sum())
```

`df.isnull()` returns a Boolean DataFrame in which `True` represents a missing value. `.sum()` counts the missing values in each column.

```

PS C:\Users\abdel> py C:\Users\abdel\Instagram_Project\etl\transform.py
Current working directory: C:\Users\abdel\Instagram_Project
Initial dataset shape: (29999, 15)

Missing values before cleaning:
post_id          0
upload_date      0
media_type       0
likes            0
comments         0
shares          0
saves            0
reach           0
impressions      0
caption_length   0
hashtags_count   0
followers_gained 0
traffic_source    0
engagement_rate  0
content_category 0
dtype: int64

```

Categorical Missing-Value Imputation

```
categorical_cols = ["media_type", "traffic_source", "content_category"]
```

Categorical fields are treated using the most frequent value, or mode.

```

for col in categorical_cols:
    mode_val = df[col].mode()[0]
    df[col] = df[col].fillna(mode_val)

```

Method	Purpose
<code>for col in categorical_cols</code>	Iterates through categorical fields
<code>.mode()</code>	Identifies the most frequent value
<code>[0]</code>	Selects the first mode when multiple modes exist
<code>.fillna(mode_val)</code>	Replaces missing values

Mode imputation preserves an existing valid category and avoids creating an artificial category such as **Unknown**. However, imputed records are assigned to the most common category and may not reflect their original unobserved value.

Numerical Missing-Value Imputation

```
numerical_cols = [  
    "likes", "comments", "shares", "saves",  
    "reach", "impressions", "followers_gained",  
    "caption_length", "hashtags_count", "engagement_rate"  
]
```

Numerical fields are imputed with their column median.

```
for col in numerical_cols:  
    median_val = df[col].median()  
    df[col] = df[col].fillna(median_val)
```

Method	Purpose
<code>for col in numerical_cols</code>	Iterates through numerical fields
<code>.median()</code>	Calculates the column median
<code>.fillna(median_val)</code>	Replaces missing values

Median imputation is appropriate for potentially skewed social-media metrics such as likes, reach, impressions, and engagement rate. Unlike the mean, the median is less affected by unusually large observations.

The script prints the imputation value for each field, creating a basic audit trail.

Data validation and type conversion

The `upload_date` field is converted to Pandas datetime format with:

```
df["upload_date"] = pd.to_datetime(df["upload_date"], errors="coerce")
```

The parameter `errors="coerce"` converts invalid dates to **NaT** (*Not a Time*) rather than stopping execution immediately.

Invalid dates are then removed:

```
before_drop = df.shape[0]
df = df.dropna(subset=["upload_date"])
after_drop = df.shape[0]
```

Operation	Purpose
<code>before_drop = df.shape[0]</code>	Stores the pre-cleaning row count
<code>.dropna(subset=["upload_date"])</code>	Removes rows with invalid or missing dates
<code>after_drop = df.shape[0]</code>	Stores the post-cleaning row count
Row-count difference	Reports the number of removed records

```
print(f"Dropped {before_drop - after_drop} rows with invalid upload_date")
```

Valid dates are required for year, month, daily, and monthly growth analysis. Removing invalid prevents records from being assigned incorrectly to reporting periods.

Categorical standardization

The script standardizes categorical fields using:

```
# ---- Standardization ----
df["media_type"] = df["media_type"].str.title().str.strip()
df["traffic_source"] = df["traffic_source"].str.title().str.strip()
df["content_category"] = df["content_category"].str.title().str.strip()
print("Standardized categorical columns")
```

Method	Purpose
<code>.str.title()</code>	Converts text to title case
<code>.str.strip()</code>	Removes leading and trailing whitespace

This prevents logically identical values such as `reel`, `REEL`, and `Reel` from being treated as separate categories. Standardization is required for accurate grouping, filtering, dimension creation, and dashboard reporting.

Duplicate removal

Duplicate records are removed using `post_id` as the business key:

```
before_dup = df.shape[0]
df = df.drop_duplicates(subset=["post_id"])
after_dup = df.shape[0]
```

Method	Purpose
<code>before_dup = df.shape[0]</code>	Stores the row count before deduplication
<code>drop_duplicates(subset=["post_id"])</code>	Keeps the first record for each post ID
<code>after_dup = df.shape[0]</code>	Stores the row count after deduplication
Row-count difference	Reports the number of removed duplicates

```
df = df.drop_duplicates(subset=["post_id"])
```

This step above prevents double-counting of interactions, reach, impressions, and follower growth. It also supports the database design in which `PostID` functions as the fact-table primary key.

```
print(f"Removed {before_dup - after_dup} duplicate rows based on post_id")
```

This line above displays the number of removed duplicate records

```
Filled missing categorical 'media_type' with mode: Carousel
Filled missing categorical 'traffic_source' with mode: Home Feed
Filled missing categorical 'content_category' with mode: Photography
Filled missing numerical 'likes' with median: 99580.0
Filled missing numerical 'comments' with median: 5042.0
Filled missing numerical 'shares' with median: 2498.0
Filled missing numerical 'saves' with median: 7483.0
Filled missing numerical 'reach' with median: 992181.0
Filled missing numerical 'impressions' with median: 1241026.0
Filled missing numerical 'followers_gained' with median: 501.0
Filled missing numerical 'caption_length' with median: 1100.0
Filled missing numerical 'hashtags_count' with median: 15.0
Filled missing numerical 'engagement_rate' with median: 9.19
Dropped 0 rows with invalid upload_date
Standardized categorical columns
Removed 0 duplicate rows based on post_id
```

Saving the cleaned dataset

```
df.to_csv(f"{processed_dir}/Instagram_Analytics_clean.csv", index=False)
```

The cleaned DataFrame is exported as `Instagram_Analytics_clean.csv`

This file contains the cleaned source fields before KPI enrichment. It provides an audit layer between the raw extraction and analytical transformation.

```
Missing values after cleaning:
```

```
post_id          0
upload_date      0
media_type       0
likes            0
comments         0
shares           0
saves            0
reach            0
impressions      0
caption_length   0
hashtags_count   0
followers_gained 0
traffic_source   0
engagement_rate  0
content_category 0
dtype: int64
Cleaned data saved
```

6.4.5 KPI Transformation and Feature Engineering

The `transform_data(df)` function receives the cleaned DataFrame and creates the fields required for reporting and dimensional modeling.

```
def transform_data(df):
```

The function performs the following operations:

- Calculates total engagement.
- Creates monthly reporting fields.
- Calculates monthly engagement growth.
- Creates the high-engagement flag.
- Calculates average engagement by media type.
- Detects engagement-rate outliers.
- Exports the KPI-enriched dataset.
- Builds dimension tables.
- Constructs the post-level fact table.

6.4.5.1 Total engagement

```
df["total_engagement"] = df["likes"] + df["comments"] + df["shares"] + df["saves"]
```

The metric is calculated as:

$$\text{Total Engagement} = \text{Likes} + \text{Comments} + \text{Shares} + \text{Saves}$$

This measure represents absolute interaction volume and supports:

- Post-level ranking.
- Category and media comparisons.
- Monthly aggregation.
- Top-post analysis.
- Portfolio-level engagement reporting.

It is not normalized by reach or impressions and therefore should not be interpreted as engagement efficiency.

Time features

```
# ---- Time Features ----
df["upload_date"] = pd.to_datetime(df["upload_date"])
df["year_month"] = df["upload_date"].dt.to_period("M")
```

The script ensures that `upload_date` is stored as a datetime field and creates a monthly period using `.dt.to_period("M")`

This produces keys such as `2024-11` and `2025-06`, which are used for monthly aggregation and growth-rate calculations. .

Monthly engagement growth rate

The script first aggregates total engagement by month:

```
monthly_engagement = df.groupby("year_month")["total_engagement"].sum().reset_index()
```

It then calculates the month-over-month percentage change using `.pct_change()`:

```
monthly_engagement["engagement_growth_rate"] = monthly_engagement["total_engagement"].pct_change()
```

The calculation is conceptually expressed as:

$$\text{Growth Rate } t = (\text{Engagement } t - \text{Engagement } t-1) / \text{Engagement } t-1$$

The monthly growth values are merged back into the post-level DataFrame:

```
df = df.merge(monthly_engagement[["year_month", "engagement_growth_rate"]],
              on="year_month", how="left")
```

Consequently, every post in the same month receives the same monthly growth value.

The first month has no preceding period, so its growth rate is expected to be null. The validation and loading logic therefore permits a null value in `engagement_growth_rate`.

This metric should be interpreted carefully when a reporting period is partial. A partial month can produce a mathematically valid but non-comparable percentage change.

High-engagement flag

The script flags posts whose total engagement exceeds the dataset-wide mean:

```
avg_engagement = df["total_engagement"].mean()
df["high_engagement_flag"] = (df["total_engagement"] > avg_engagement).astype(int)
```

Value	Meaning
1	Total engagement is above the dataset mean
0	Total engagement is equal or below the dataset mean

This field supports the percentage of above-average posts displayed in Power BI.

The flag is relative to the dataset and is not an external business benchmark. Because the threshold is based on the arithmetic mean, unusually large values can raise the classification threshold.

Average engagement by media type

The script calculates the mean total engagement for each media format:

```
avg_engagement_media = (
    df.groupby("media_type")["total_engagement"]
      .mean()
      .reset_index()
      .rename(columns={"total_engagement": "avg_engagement_by_media"})
)
```

The result is merged back into each post-level data:

```
df = df.merge(avg_engagement_media, on="media_type", how="left")
```

Each post therefore contains a reference value representing the average total engagement for its media type. This supports comparison between an individual post and the average performance of its format.

Engagement-rate outlier detection

The script identifies unusual values in `engagement_rate` using the Interquartile Range method.

```
Q1 = df["engagement_rate"].quantile(0.25)
Q3 = df["engagement_rate"].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
```

The IQR represents the spread of the central 50% of observations:

$$\text{IQR} = \text{Q3} - \text{Q1}$$

The conventional bounds are:

$$\text{Lower Bound} = \text{Q1} - 1.5 (\text{IQR})$$

$$\text{Upper Bound} = \text{Q3} + 1.5 (\text{IQR})$$

Observations outside these bounds are flagged:

```
df["is_outlier"] = (
    ~df["engagement_rate"].between(lower_bound, upper_bound)
).astype(int)
```

Value	Meaning
0	Engagement rate lies within the IQR bounds
1	Engagement rate lies outside the IQR bounds

The `.between()` method identifies values within the bounds. The `~` operator reverses that Boolean result so that observations outside the interval are selected. `.astype(int)` converts the Boolean flag into a numeric field suitable for CSV export, MySQL storage, Power BI filtering, and DAX calculations.

The script prints the calculated limits and number of flagged records, providing transparency around the outlier-detection result.

The outlier flag does not indicate that a post is incorrect. It identifies a statistically unusual engagement-rate observation. Such records may represent genuine exceptional performance, low exposure denominators, campaigns, or data-quality issues. The project therefore retains them and compares them separately rather than deleting them.

Saving the KPI-enriched dataset

The transformed DataFrame is exported as:

`Instagram_Analytics_clean_with_KPIs.csv`

It contains the cleaned source fields plus the derived KPI and analytical fields.

6.4.6 Dimensional Model Construction

The transformation stage creates a star-schema-oriented structure consisting of dimension tables and a central fact table.

- The **Fact table** stores post-level events, numerical measures, derived KPIs, and foreign keys.
- The **Dimension tables** store descriptive attributes used for filtering, grouping, and analysis.

Each record in `instagram_fact` represents one Instagram post. The dimensions provide descriptive information about date, content, media format, and traffic source.

Time dimension

The time dimension is created from distinct publication dates:

```
time_dim = df[["upload_date"]].drop_duplicates()
time_dim["year"] = time_dim["upload_date"].dt.year
time_dim["month"] = time_dim["upload_date"].dt.month
time_dim["day"] = time_dim["upload_date"].dt.day
```

Field	Description
<code>upload_date</code>	Post publication date
<code>year</code>	Calendar year

month	Numeric month
day	Day of month

The table is exported as `time_dim.csv`

Content dimension

The content dimension is created from unique combinations of media type, content category, caption length, and hashtag count:

```
content_dim = df[
    ["media_type", "content_category", "caption_length", "hashtags_count"]
].drop_duplicates().reset_index(drop=True)

content_dim["content_id"] = content_dim.index + 1
```

A surrogate `content_id` is generated by resetting the index and adding one. This creates identifiers beginning at 1 rather than 0.

The table is exported as `content_dim.csv`

Media dimension

The media dimension contains one record for each distinct media type:

```
media_dim = df[["media_type"]].drop_duplicates().reset_index(drop=True)
media_dim["media_id"] = media_dim.index + 1
```

Each record receives a surrogate `media_id`

The table is exported as `media_dim.csv`

Traffic dimension

The traffic dimension contains one record for each distinct traffic source:

```
traffic_dim = df[["traffic_source"]].drop_duplicates().reset_index(drop=True)
traffic_dim["traffic_id"] = traffic_dim.index + 1
```

It is exported as `traffic_dim.csv`

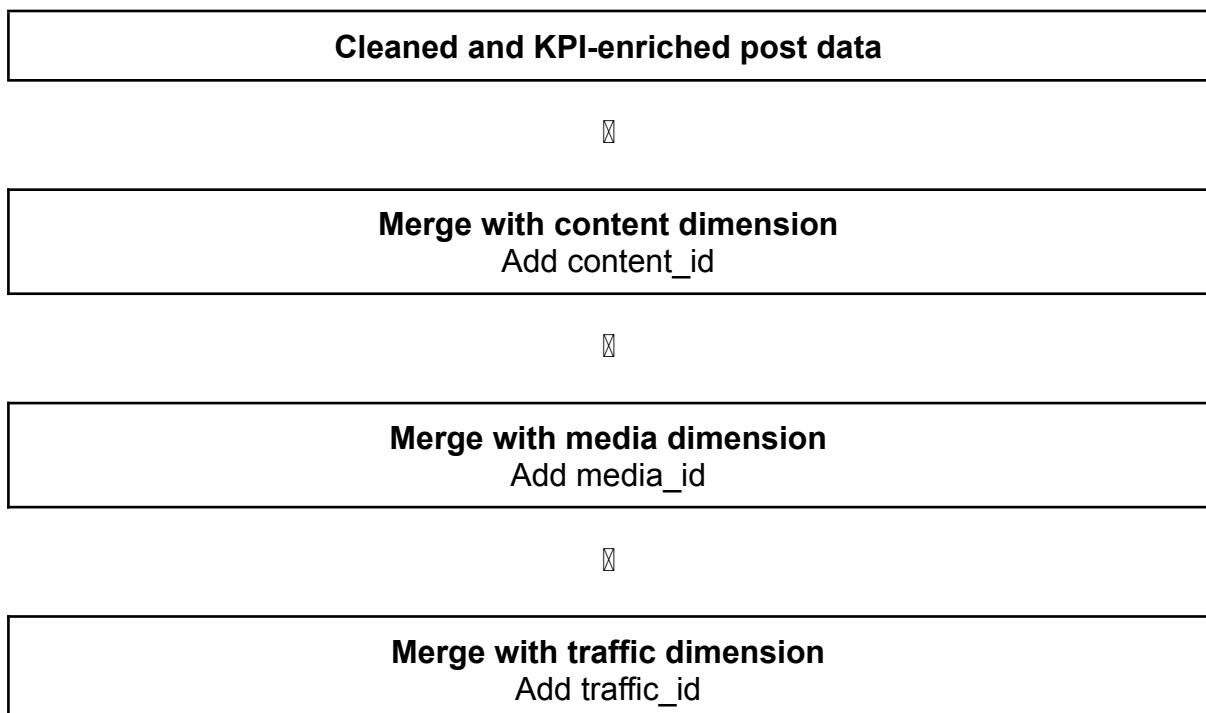
6.4.7 Fact Table Construction

Dimensions Merge

The `instagram_fact` table is created by combining the cleaned post data, derived KPIs, and dimension identifiers.

```
# ----- Fact Table -----
instagram_fact = df.merge(
    content_dim,
    on=["media_type", "content_category", "caption_length", "hashtags_count"],
    how="left"
).merge(
    media_dim[['media_type', 'media_id']],
    on='media_type',
    how='left'
).merge(
    traffic_dim[['traffic_source', 'traffic_id']],
    on='traffic_source',
    how='left'
)
```

The process is:



The merges use `how="left"`:

```
how="left"
```

A left join retains every record from the enriched post-level dataset and appends matching dimension keys when available. This prevents valid fact records from being removed during lookups.

Fact-table fields

Field group	Included fields
Post identifier	<code>post_id</code>
Time	<code>upload_date</code>
Dimension keys	<code>content_id, media_id, traffic_id</code>
Interaction metrics	<code>likes, comments, shares, saves</code>
Exposure metrics	<code>reach, impressions</code>
Audience growth	<code>followers_gained</code>
Rate metric	<code>engagement_rate</code>
Derived KPIs	<code>total_engagement, engagement_growth_rate, high_engagement_flag, avg_engagement_by_media, is_outlier</code>
Source attribute	<code>traffic_source</code>

The result is exported as `instagram_fact.csv`

Handling missing dimension identifiers

```
# Optional: fill NaN IDs
instagram_fact['traffic_id'] = instagram_fact['traffic_id'].fillna(0).astype(int)
instagram_fact['media_id'] = instagram_fact['media_id'].fillna(0).astype(int)
```

Missing dimension identifiers are replaced with `0` and converted to integers:

- `.fillna(0)` replaces missing traffic and media IDs with `0`.
- `.astype(int)` converts the resulting column to integer type.

This prevents `NaN` values from causing loading errors or violating integer-field constraints.

A value of `0` acts as a placeholder for an unmatched dimension value. It is not a formally created `Unknown` member. A production warehouse would normally create an explicit `Unknown` record in each dimension and assign it key `0`.

Fact-table fields selection and export

```
instagram_fact = instagram_fact[[
    "post_id", "upload_date", "content_id", "media_id", "traffic_id",
    "likes", "comments", "shares", "saves",
    "reach", "impressions", "followers_gained", "engagement_rate",
    "total_engagement", "engagement_growth_rate",
    "high_engagement_flag", "avg_engagement_by_media", "is_outlier",
    "traffic_source"
]]
instagram_fact.to_csv(f"{processed_dir}/instagram_fact.csv", index=False)
print("Fact table with KPIs, media_id, and traffic_id saved")

return df, instagram_fact
```

The script restricts the DataFrame to the fields required by the target fact table.

```
--- Adding KPI columns ---
Added 'total_engagement' column
Added 'engagement_growth_rate' column
Added 'high_engagement_flag' column
Added 'avg_engagement_by_media' column
Flagged 2557 outlier posts out of 29999 (IQR bounds: -10.5550 - 31.4050)
Cleaned dataset with KPIs saved
Fact table with KPIs, media_id, and traffic_id saved
```

The selected fact table is then exported for database loading.

6.4.8 Exploratory Analytical Outputs

```
def analyze_data(df):
    print("\n--- Running Engagement Driver Analysis ---")
```

The `analyze_data(df)` function produces two supplementary analytical files:

1. A correlation report based on one-hot-encoded category and traffic-source variables.
2. A Category-level comparison of engagement rate before and after outlier exclusion.

This stage is exploratory and descriptive. It does not build a predictive model, test statistical significance, or establish causality.

Categorical Correlation Analysis

```
dummies = pd.get_dummies(  
    df[["content_category", "traffic_source"]]  
)
```

The script converts `content_category` and `traffic_source` into binary indicator variables using **one-hot encoding** (a standard Pearson correlation coefficient calculated between numerical variables).

For example, a Beauty record may be represented as:

Field	Value
<code>content_category_Beauty</code>	1
<code>content_category_Fashion</code>	0
<code>content_category_Travel</code>	0

The script then calculates the correlation between each indicator and `engagement_rate`:

```
corr = dummies.corrwith(df["engagement_rate"]).sort_values(  
    key=abs,  
    ascending=False  
)
```

`.corrwith()` calculates a Pearson correlation coefficient between each binary indicator and the numerical engagement-rate field. Results are sorted by absolute coefficient magnitude so that the strongest associations appear first.

The output is saved as `engagement_correlation_report.csv`

Correlation results

```
--- Running Engagement Driver Analysis ---  
Correlation with engagement_rate (top 5 by |r|):  
content_category_Beauty      0.013871  
traffic_source_External      -0.007687  
content_category_Travel      -0.007391  
traffic_source_Explore        0.005683  
content_category_Technology   -0.005551  
dtype: float64
```

The one-hot-encoded correlation analysis found **extremely weak marginal linear associations** between content category or traffic source and engagement rate.

The largest positive coefficient was observed for **Beauty**: $r = 0.0139$

The largest negative coefficient was observed for **External traffic**: $r = -0.0077$

All coefficients were below: $|r| < 0.014$

This indicates that neither content category nor traffic source has a meaningful standalone linear relationship with engagement rate in the analyzed dataset.

The result should be interpreted as a descriptive diagnostic, not as evidence that category or traffic source has no possible predictive or business relevance. The analysis evaluates only unadjusted, pairwise linear association and does not account for potential interactions or confounding factors such as media type, exposure, posting time, content quality, paid promotion, account characteristics, or campaign context.

Outlier-adjusted category comparison

The script compares category-level engagement rates using:

- All posts in each category.
- Non-outlier posts only.
- The number of flagged outliers in each category.

```
category_check = df.groupby("content_category").apply(
    lambda g: pd.Series({
        "mean_er_all": g["engagement_rate"].mean(),
        "mean_er_no_outliers": g.loc[
            g["is_outlier"] == 0,
            "engagement_rate"
        ].mean(),
        "n_outliers": g["is_outlier"].sum()
    })
).reset_index()
```

The output contains:

Field	Description
content_category	Category being analyzed
mean_er_all	Mean engagement rate using all posts
mean_er_no_outliers	Mean engagement rate where <code>is_outlier = 0</code>
n_outliers	Number of flagged observations

The output is exported as `category_outlier_check.csv`

This comparison determines whether a category's apparent performance is stable or disproportionately influenced by extreme observations.

The code uses `.groupby().apply()`:

- `.apply()` runs a function for each category group.
- `lambda` defines the calculation applied to each group.
- `pd.Series()` returns the three calculated values.
- `.loc[...]` selects the non-outlier rows.
- Summing the binary `is_outlier` field counts outliers.
- `.reset_index()` converts the grouped index into a standard column.

If every observation in a category were flagged, the non-outlier mean would be null because no observations would remain. The current script does not include a separate fallback for that case.

At approximately 30,000 rows and ten categories, the performance impact of `groupby().apply()` is negligible. For substantially larger datasets, built-in aggregation methods could be more efficient.

Category outlier-check results

The `category_outlier_check.csv` output compares average engagement rate for each content category using both the full dataset and the non-outlier population. It also reports the number of engagement-rate outliers identified within each category.

Category engagement rate – all posts vs. outliers removed:				
	content_category	mean_er_all	mean_er_no_outliers	n_outliers
0	Beauty	15.664385	9.958705	266.0
1	Comedy	14.396149	9.605303	244.0
2	Fashion	14.523533	9.926088	263.0
3	Fitness	13.948752	9.687223	260.0
4	Food	13.988711	10.229287	233.0
5	Lifestyle	14.692105	10.084330	280.0
6	Music	14.488062	9.905567	244.0
7	Photography	14.816511	9.966915	286.0
8	Technology	13.927650	10.040108	255.0
9	Travel	13.757692	10.062721	226.0

All categories experience a reduction in average engagement rate after outlier exclusion. However, the size of the reduction differs across categories.

Beauty shows the largest decline, falling from 15.66% to 9.96%, a reduction of 5.71 percentage points. In contrast, Travel and Food show the smallest reductions, decreasing by 3.69 and 3.76 percentage points, respectively.

The results indicate that raw category averages are influenced by unusually high engagement-rate observations in every category, but the magnitude of that influence is not identical across categories.

The raw category range is: $15.66\% - 13.76\% = 1.91$ percentage points

After excluding outliers, the range decreases to: $10.23\% - 9.61\% = 0.62$ percentage points

This reduction shows that the apparent performance gaps between categories are substantially compressed after outlier treatment.

Important interpretation to retain

The outlier-adjusted category comparison does not show that category is irrelevant. Instead, it shows that the raw category ranking is not stable enough to be treated as a strong standalone indicator of typical engagement-rate performance.

6.4.9 Transformation Validation

The `validate_data(df, fact)` function provides a basic data-quality gate before loading.

```
def validate_data(df, fact):  
    print("\n--- Validating Data ---")
```

The validation process:

1. Identifies fact-table columns containing null values.
2. Permits nulls only in `engagement_growth_rate`.
3. Raises an `AssertionError` if any other unexpected null exist.

```
allowed_nulls = ["engagement_growth_rate"]  
null_cols = fact.columns[fact.isnull().any()].tolist()  
unexpected = [c for c in null_cols if c not in allowed_nulls]  
  
assert len(unexpected) == 0, f"Unexpected NULLs: {unexpected}"
```

The first reporting month has no preceding month, so a null growth rate is expected. Other fact-table fields should be complete before loading.

This validation prevents incomplete records from being silently inserted into MySQL.

```
--- Validating Data ---  
Data validation completed successfully.  
PS C:\Users\abdel>
```

6.4.10 Pipeline Orchestration

```
# ----- Main -----
def main():
    df_clean = clean_data()
    df_kpi, fact = transform_data(df_clean)
    analyze_data(df_kpi)
    validate_data(df_kpi, fact)

if __name__ == "__main__":
    main()
```

`main()` executes the transformation pipeline in dependency order:

`clean_data()`

`transform_data()`

`analyze_data()`

`validate_data()`

Cleaning must occur before transformation, the fact table must exist before validation, and analytical outputs are generated from the enriched data.

6.4.11 Transformation outputs

The transformation stage creates the following outputs in `data/processed`.

Output file	Purpose
<code>Instagram_Analytics_clean.csv</code>	Cleaned source data before KPI enrichment
<code>Instagram_Analytics_clean_with_KPIs.csv</code>	Cleaned data with derived KPI fields

<code>time_dim.csv</code>	Publication-date dimension
<code>content_dim.csv</code>	Content-attribute dimension
<code>media_dim.csv</code>	Media-type dimension
<code>traffic_dim.csv</code>	Traffic-source dimension
<code>instagram_fact.csv</code>	Post-level fact table
<code>engagement_correlation_report.csv</code>	Exploratory category/source correlations
<code>category_outlier_check.csv</code>	Category engagement comparison with and without outliers

The pipeline produces nine processed files: seven data-model files and two exploratory analysis files.

6.4.12 Key methods and functions

This section provides a consolidated explanation of the main Python and Pandas methods used throughout the script.

Method or Function	Role in the transformation process
<code>os.chdir()</code>	Changes the current working directory
<code>os.getcwd()</code>	Retrieves the current working directory
<code>os.makedirs()</code>	Creates output directories
<code>pd.read_csv()</code>	Reads CSV data
<code>df.shape</code>	Returns row and column count
<code>df.isnull()</code>	Identifies missing values
<code>.sum()</code>	Counts missing values or aggregates measures
<code>.mode()</code>	Calculates the most frequent categorical value
<code>.fillna()</code>	Replaces missing values
<code>.median()</code>	Calculates a numerical median
<code>pd.to_datetime()</code>	Converts values to datetime format
<code>.dropna()</code>	Removes rows with selected missing values

<code>.str.title()</code>	Standardizes text capitalization
<code>.str.strip()</code>	Removes surrounding whitespace
<code>.drop_duplicates()</code>	Removes duplicate records
<code>.to_csv()</code>	Exports a DataFrame to CSV format
<code>.dt.year</code>	Extracts the year from datetime field
<code>.dt.month</code>	Extracts the month from datetime field
<code>.dt.day</code>	Extracts the day from datetime field
<code>.dt.to_period("M")</code>	Creates monthly periods
<code>.groupby()</code>	Groups records for aggregation
<code>.mean()</code>	Calculates the arithmetic means
<code>.pct_change()</code>	Calculates relative change
<code>.merge()</code>	Combines DataFrames
<code>.quantile()</code>	Calculates percentiles
<code>.between()</code>	Tests whether values lie within an interval
<code>.astype(int)</code>	Converts values to integers
<code>pd.get_dummies()</code>	Creates binary indicator variables
<code>.corrwith()</code>	Calculates correlations
<code>.sort_values()</code>	Sorts results
<code>.head()</code>	Returns initial records
<code>.apply()</code>	Applies a function to groups
<code>.loc[]</code>	Selects rows using labels or conditions
<code>.tolist()</code>	Converts results to a Python list
<code>assert</code>	Stops execution when validation fails

6.5 Configuration Management

The `.env` file stores database configuration separately from the Python scripts.

```

DB_HOST=localhost
DB_USER=root
DB_PASSWORD=
DB_NAME=instagram_db

```

This prevents credentials from being hard-coded and allows the same code to be reused across environments.

Variable	Purpose
DB_HOST	MySQL server address
DB_USER	Database username
DB_PASSWORD	Database password
DB_NAME	Target database name

The loading script retrieves these values through environment-variable loading.

6.6 Loading Stage

The `Load.py` script implements the loading phase. It creates the MySQL schema and loads the processed dimension and fact files created during transformation.

The script uses:

- Python.
- Pandas.
- MySQL Connector/Python.
- `python-dotenv`.
- `os`.

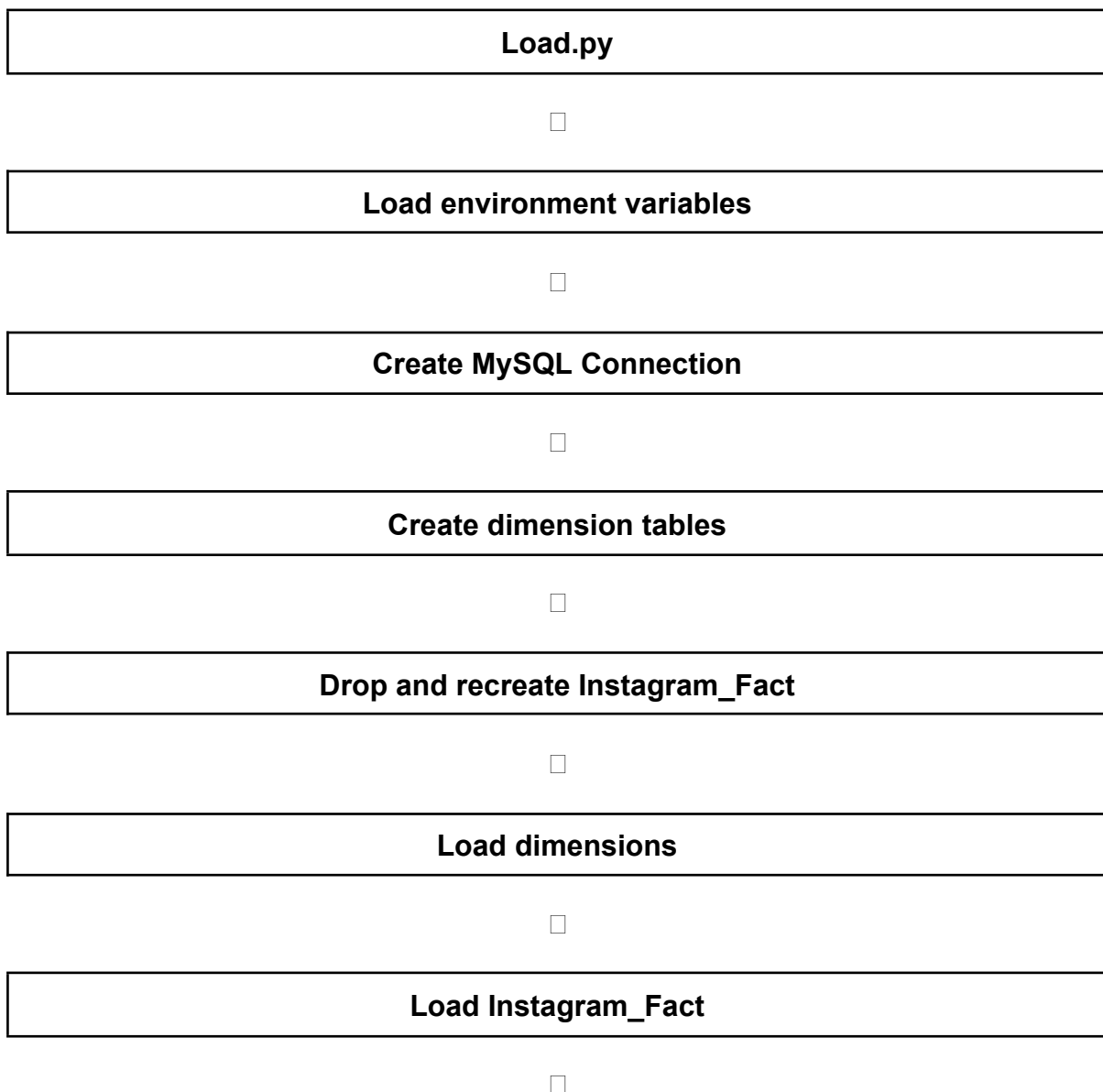
The loader follows a dimension-first strategy: `Time_Dim`, `Content_Dim`, `Media_Dim`, and `Traffic_Dim` are loaded before `Instagram_Fact`. This ensures that referenced dimension keys exist before fact records are inserted.

6.6.1 Load Workflow

Stage	Description
-------	-------------

Load configuration	Retrieves database settings from environment variables
Create connection	Connects to MySQL
Create schema	Creates dimensions and recreates the fact table
Load dimensions	Inserts time, content, media, and traffic records
Load fact data	Inserts post-level metrics and KPIs
Commit transactions	Saves changes
Close connection	Releases the database connection

The process can be represented as follows:



Commit and close connection

6.6.2 Libraries and Tools

The script imports the following modules:

```
#Load.py

import pandas as pd
import mysql.connector
from mysql.connector import Error
from dotenv import load_dotenv
import os
```

Library or module	Role
pandas	Reads processed CSV files and identifies missing values
mysql.connector	Connects Python to MySQL
Error	Handles MySQL connection exceptions
dotenv	Loads database credentials and configuration variables from .env
os	Retrieves environment variables and builds portable file paths

Pandas reads each processed file and checks values with `pd.isna()`. Missing Pandas values are converted to Python `None`, which MySQL stores as `NULL`.

MySQL Connector/Python provides the connection, cursor, SQL execution, transaction, and closing operations.

`python-dotenv` keeps database credentials outside the code. This is safer for shared, version-controlled, or deployed projects.

6.6.3 Database Configuration

```
# Load environment variables
load_dotenv()
DB_HOST = os.getenv("DB_HOST")
DB_USER = os.getenv("DB_USER")
DB_PASSWORD = os.getenv("DB_PASSWORD")
DB_NAME = os.getenv("DB_NAME")
```

Enironment variable	Purpose
DB_HOST	MySQL server location
DB_USER	Authentication username
DB_PASSWORD	Authentication password
DB_NAME	Target database name

Separating configuration from code improves security, portability, and maintainability.

The `DB_HOST` value may refer to a local MySQL instance, such as localhost, or a remote server address, depending on the deployment environment.

6.6.4 Column Mappings

The `column_mappings` dictionary aligns lowercase processed-file fields with the PascalCase naming convention used in MySQL.

The general mapping structure is:

```
{ "TableName": { "CSVColumn": "MySQLColumn" } }
```

Before loading, the relevant DataFrame columns are renamed:

```
df.rename(columns=column_mappings[table_name], inplace=True)
```

This ensures that CSV field names match the target database schema.

Time dimension mapping

```
# Column mappings for CSV → MySQL
column_mappings = {
    "Time_Dim": {
        "upload_date": "UploadDate",
        "year": "Year",
        "month": "Month",
        "day": "Day"
    },

```

CSV field	MySQL field	Description
upload_date	UploadDate	Publication date
year	Year	Extracted year
month	Month	Extracted month
day	Day	Extracted day of month

Content dimension mapping

```
"Content_Dim": {
    "content_id": "ContentID",
    "media_type": "MediaType",
    "content_category": "ContentCategory",
    "caption_length": "CaptionLength",
    "hashtags_count": "HashtagsCount"
},
```

CSV field	MySQL field	Description
content_id	ContentID	Content surrogate key
media_type	MediaType	Content format
content_category	ContentCategory	Content category
caption_length	CaptionLength	Caption-length measure
hashtags_count	HashtagsCount	Hashtag count

Media dimension mapping

```
"Media_Dim": {
  "media_id": "MediaID",
  "media_type": "MediaType"
},
```

CSV field	MySQL field	Description
media_id	MediaID	Media surrogate key
media_type	MediaType	Media format

Traffic dimension mapping

```
"Traffic_Dim": {
  "traffic_id": "TrafficID",
  "traffic_source": "TrafficSource"
},
```

CSV field	MySQL field	Description
traffic_id	TrafficID	Traffic surrogate key
traffic_source	TrafficSource	Traffic source

Fact-table mapping

The fact-table mapping includes post identifiers, foreign keys, performance measures, and derived KPI fields.

```

    "Instagram_Fact": {
      "post_id": "PostID",
      "upload_date": "UploadDate",
      "content_id": "ContentID",
      "media_id": "MediaID",
      "traffic_id": "TrafficID",
      "likes": "Likes",
      "comments": "Comments",
      "shares": "Shares",
      "saves": "Saves",
      "reach": "Reach",
      "impressions": "Impressions",
      "followers_gained": "FollowersGained",
      "engagement_rate": "EngagementRate",
      "total_engagement": "TotalEngagement",
      "engagement_growth_rate": "EngagementGrowthRate",
      "high_engagement_flag": "HighEngagementFlag",
      "avg_engagement_by_media": "AvgEngagementByMedia",
      "is_outlier": "IsOutlier",
      "traffic_source": "TrafficSource"
    }
  }
}

```

CSV field	MySQL field	Analytical role
post_id	PostID	Fact-table primary key
upload_date	UploadDate	Date key
content_id	ContentID	Content foreign key
media_id	MediaID	Media foreign key
traffic_id	TrafficID	Traffic foreign key
likes	Likes	Recorded likes
comments	Comments	Recorded comments
shares	Shares	Recorded shares
saves	Saves	Recorded saves

reach	Reach	Recorded reach
impressions	Impressions	Recorded impressions
followers_gained	FollowersGained	Follower acquisition
engagement_rate	EngagementRate	Source engagement rate
total_engagement	TotalEngagement	Derived interaction volume
engagement_growth_rate	EngagementGrowthRate	Monthly growth metric
high_engagement_flag	HighEngagementFlag	Above-average total engagement flag
is_outlier	IsOutlier	Engagement-rate outlier flag
traffic_source	TrafficSource	Retained descriptive field

Keeping both `TrafficID` and `TrafficSource` in the fact table is not strictly necessary in a fully normalized star schema, because the label can be retrieved from `Traffic_Dim`. However, retaining the descriptive field may simplify selected queries and reporting operations.

6.6.5 MySQL connection

The `create_connection()` function establishes a reusable database connection.

```
def create_tables(connection):
```

The connection uses the environment variables:

```
try:
    connection = mysql.connector.connect(
        host=DB_HOST,
        user=DB_USER,
        password=DB_PASSWORD,
        database=DB_NAME
    )
```

Parameter	Purpose
-----------	---------

host	MySQL server address
user	Authentication username
password	Authentication password
database	Target database

The connection is tested inside a `try` block. If it fails, the `except Error as e` handler captures the database error.

```
if connection.is_connected():
    print("Connected to MySQL database")
    return connection
```

```
except Error as e:
    print(f"Error creating connection: {e}")
    return None
```

Potential causes include:

- Incorrect credentials.
- An unavailable MySQL server.
- An invalid database name.
- Network connectivity issues.
- Insufficient user permissions.
- Incorrect host or port configuration.

Returning `None` prevents the loader from proceeding without a valid connection.

6.6.6 MySQL schema creation

The `create_tables(connection)` function creates the warehouse tables.

```
# ----- Create Tables -----
def create_tables(connection):
    sql = ""
```

The schema includes:

- `Time_Dim`
- `Content_Dim`

- Media_Dim
- Traffic_Dim
- Instagram_Fact

Dimension tables are created before the fact table because the fact table references their keys.

The variable `sql` contains multiple SQL statements in a multiline Python string.

Time dimension

```
CREATE TABLE IF NOT EXISTS Time_Dim (  
    UploadDate DATE PRIMARY KEY,  
    Year INT,  
    Month INT,  
    Day INT  
);
```

`UploadDate` acts as the time-dimension primary key and is referenced by the fact table. `IF NOT EXISTS` prevents an error when the table already exists.

Content dimension

```
CREATE TABLE IF NOT EXISTS Content_Dim (  
    ContentID INT PRIMARY KEY,  
    MediaType VARCHAR(50),  
    ContentCategory VARCHAR(50),  
    CaptionLength INT,  
    HashtagsCount INT  
);
```

`ContentID` is the primary key for the content dimension and links posts to their content attributes.

Media dimension

```
CREATE TABLE IF NOT EXISTS Media_Dim (  
    MediaID INT PRIMARY KEY,  
    MediaType VARCHAR(50)  
);
```

This table stores the reference values for Photo, Video, Reel, and Carousel posts.

Traffic dimension

```
CREATE TABLE IF NOT EXISTS Traffic_Dim (  
    TrafficID INT PRIMARY KEY,  
    TrafficSource VARCHAR(100)  
);
```

This table stores the descriptive traffic-source values.

Instagram fact table

The existing fact table is explicitly removed before recreation:

```
DROP TABLE IF EXISTS Instagram_Fact;
```

A new post-level fact table is then created:

```

CREATE TABLE IF NOT EXISTS Instagram_Fact (
    PostID VARCHAR(50) PRIMARY KEY,
    UploadDate DATE,
    ContentID INT,
    MediaID INT,
    TrafficID INT,
    Likes INT,
    Comments INT,
    Shares INT,
    Saves INT,
    Reach INT,
    Impressions INT,
    FollowersGained INT,
    EngagementRate FLOAT,
    TotalEngagement INT,
    EngagementGrowthRate FLOAT,
    HighEngagementFlag TINYINT,
    AvgEngagementByMedia FLOAT,
    IsOutlier TINYINT,
    TrafficSource VARCHAR(100),
    FOREIGN KEY (ContentID) REFERENCES Content_Dim(ContentID),
    FOREIGN KEY (UploadDate) REFERENCES Time_Dim(UploadDate),
    FOREIGN KEY (MediaID) REFERENCES Media_Dim(MediaID),
    FOREIGN KEY (TrafficID) REFERENCES Traffic_Dim(TrafficID)
);

```

Instagram_Fact stores one record per post and contains the measures and derived fields used by Power BI.

Field group	Examples
Primary identifier	PostID
Foreign keys	UploadDate, ContentID, MediaID, TrafficID
Interaction metrics	Likes, Comments, Shares, Saves
Exposure metrics	Reach, Impressions
Audience growth	FollowersGained

Rate metric	EngagementRate
Derived KPIs	TotalEngagement, EngagementGrowthRate, HighEngagementFlag, AvgEngagementByMedia, IsOutlier

`HighEngagementFlag` and `IsOutlier` use the `TINYINT` because they store binary values:

Value	Meaning
0	False or not flagged
1	True or flagged

Executing table-creation statements

```

cursor = connection.cursor()
for stmt in sql.split(";"):
    if stmt.strip():
        cursor.execute(stmt)
connection.commit()
cursor.close()
print("Tables created successfully")

```

The SQL string is split at semicolons so that each statement can be executed separately. `.strip()` removes surrounding whitespace, and the conditional check prevents empty statements from being sent to MySQL.

The transaction is committed with `connection.commit()`.

The cursor is then closed.

Refresh Behaviour

The loader applies different refresh rules:

- Dimension tables use `CREATE TABLE IF NOT EXISTS`.
- The fact table is dropped and recreated during each run.
- Dimension records are inserted with `INSERT IGNORE`.

This creates a full refresh of the fact table but not of the dimensions. Existing dimension records are retained, and duplicate inserts are ignored. Existing dimension attributes are not updated.

This is suitable for a stable initial dataset. A production pipeline with changing dimension attributes would require explicit refresh or upsert logic, such as:

```
INSERT ... ON DUPLICATE KEY UPDATE
```

6.6.7 Loading CSV data

The reusable `load_data()` function loads one processed CSV file into a target MySQL table.

```
def load_data(connection, table_name, csv_file):
```

Parameter	Description
<code>connection</code>	Active MySQL connection
<code>table_name</code>	Target table
<code>csv_file</code>	Processed CSV path

The function:

1. Reads the CSV into a DataFrame.
2. Applies the relevant column mapping.
3. Creates a cursor.
4. Builds a parameterized insert statement.
5. Converts rows into tuples.
6. Converts Pandas missing values to `None`.
7. Inserts the rows.
8. Commits the transaction.
9. Closes the cursor.
10. Displays a completion message.

Reading and renaming fields

```
df = pd.read_csv(csv_file)
```

The processed file is loaded into a DataFrame. If a mapping exists, the columns are renamed:

```
if table_name in column_mappings:
    df.rename(columns=column_mappings[table_name], inplace=True)
```

`inplace=True` modifies the existing DataFrame rather than creating a second object.

Parameterized insert statement

The script creates the database-column list:

```
cursor = connection.cursor()
cols = ", ".join(df.columns)
```

It creates one `%s` placeholder per field:

```
vals = ", ".join(["%s"] * len(df.columns))
```

The resulting query is:

```
query = f"INSERT IGNORE INTO {table_name} ({cols}) VALUES ({vals})"
```

Parameterized queries separate SQL structure from data values. This improves data handling and reduces formatting problems and SQL-injection risk for inserted values.

INSERT IGNORE behaviour

`INSERT IGNORE` allows MySQL to continue processing when certain insert conflicts occur, particularly duplicate primary-key records in dimension tables.

However, it should be used carefully:

- It prevents some duplicate or conversion errors from stopping the load.
- It does not update existing records.
- It may suppress warnings.
- It is not a complete upsert strategy.

For production incremental loading, `INSERT ... ON DUPLICATE KEY UPDATE` may appropriate update behaviour.

Converting rows to tuples

```
for row in df.itertuples(index=False, name=None):
```

The script iterates through DataFrame rows.

Argument	Purpose
<code>index=False</code>	Excludes the DataFrame index
<code>name=None</code>	Returns standard Python tuples

Missing Pandas values are converted to Python `None`:

```
row = tuple(None if pd.isna(value) else value for value in row)
```

The database connector interprets `None` as SQL `NULL`. This is required for fields such as the first-month's `EngagementGrowthRate`, which has no previous period.

Executing and committing inserts

Rows are inserted through the cursor:

```
cursor.execute(query, row)
```

After insertion, the transaction is committed:

```
connection.commit()
```

The cursor is closed and a completion message is displayed:

```
cursor.close()
print(f"Loaded {table_name}")
```

6.6.8 Main Load Workflow

`main()` orchestrates the loading stage.

```
def main():
```

It first attempts to create a database connection:

```
connection = create_connection()
```

The remaining operations execute only when the connection is valid:

```
if connection:
    try:
        create_tables(connection)
```

The script creates the schema and resolves the project root:

```
base_dir = os.path.abspath(
    os.path.join(os.path.dirname(__file__), "..")
)
```

This allows processed file paths to be built relative to the project structure rather than the active terminal directory.

```
load_data(connection, "Time_Dim", os.path.join(base_dir, "data", "processed", "time_dim.csv"))
load_data(connection, "Content_Dim", os.path.join(base_dir, "data", "processed", "content_dim.csv"))
load_data(connection, "Media_Dim", os.path.join(base_dir, "data", "processed", "media_dim.csv"))
load_data(connection, "Traffic_Dim", os.path.join(base_dir, "data", "processed", "traffic_dim.csv"))
load_data(connection, "Instagram_Fact", os.path.join(base_dir, "data", "processed", "instagram_fact.csv"))
```

The loading order is:

Order	Target table	Source file
1	Time_Dim	data/processed/time_dim.csv
2	Content_Dim	data/processed/content_dim.csv
3	Media_Dim	data/processed/media_dim.csv
4	Traffic_Dim	data/processed/traffic_dim.csv
5	Instagram_Fact	data/processed/instagram_fact.csv

Dimensions are loaded first so that fact-table keys reference existing dimension records.

The `finally` block closes the connection after processing, including when an error occurs:

```
finally:
    if connection.is_connected():
        connection.close()
        print("Database connection closed")
```

Finally:

```
if __name__ == "__main__":
    main()
```

ensures that loading runs when `Load.py` is executed directly but not automatically when the file is imported as a module.

6.6.9 Loading Stage Result

```
PS C:\Users\abdel>
py C:\Users\abdel\Instagram_Project\etl\Load.py
Connected to MySQL database
Tables created successfully
Loaded Time_Dim
Loaded Content_Dim
Loaded Media_Dim
Loaded Traffic_Dim
Loaded Instagram_Fact
Database connection closed
PS C:\Users\abdel>
```

After successful execution, the MySQL warehouse contains:

Table	Role
Time_Dim	Publication-date attributes
Content_Dim	Content classification and publishing attributes
Media_Dim	Media-format reference values
Traffic_Dim	Traffic-source reference values
Instagram_Fact	Post-level metrics, KPIs, and dimension keys

The completed model provides the structured data layer for SQL analysis and Power BI reporting. It supports filtering and aggregation by date, content category, media type, traffic source, exposure, engagement, and outlier status.

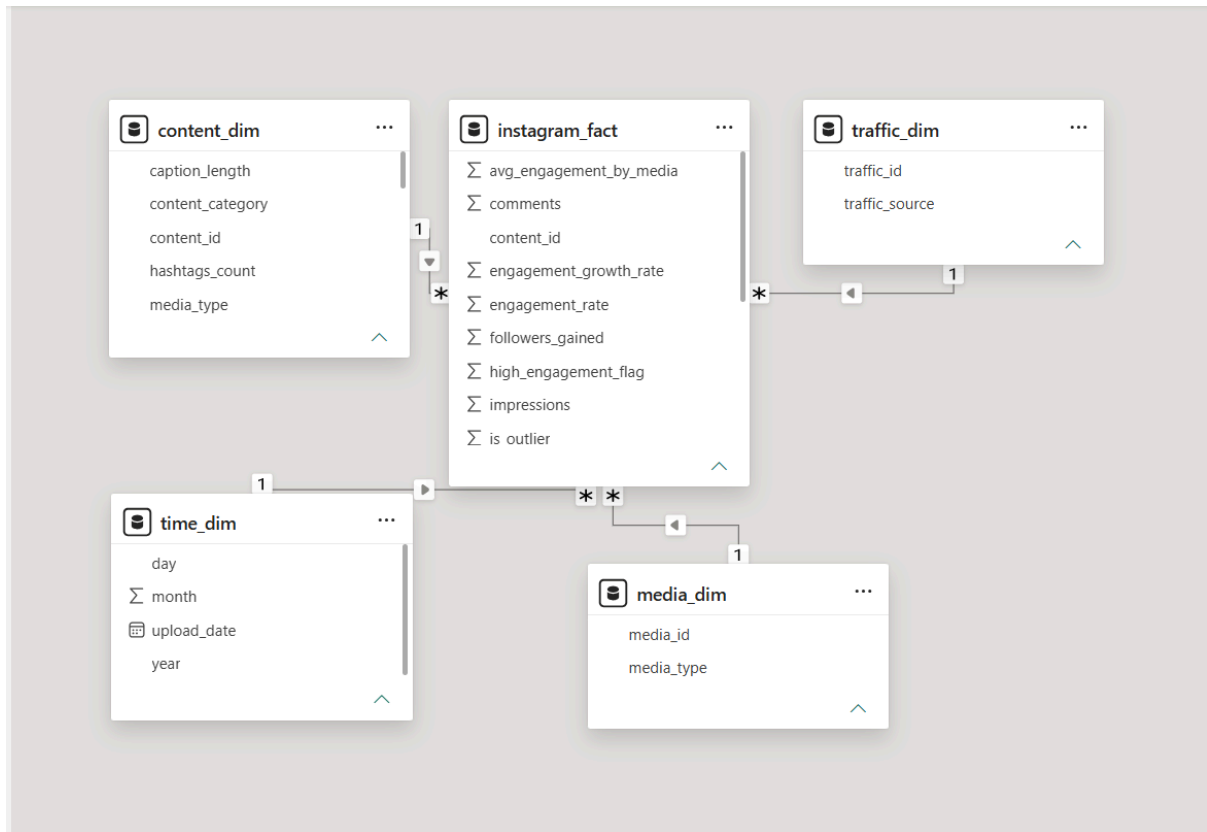
6.7 ETL Controls and Reproducibility

The ETL implementation includes several controls that improve reliability:

- Dynamic path resolution avoids dependence on a specific execution directory.
- Separate staging and processed folders preserve data lineage.
- Missing-value treatment is documented through printed imputation values.
- Invalid dates are reported and removed before time analysis.
- Duplicate `post_id` records are counted and removed.
- KPI calculations are generated consistently in Python.
- The IQR limits and outlier count are printed for auditability.
- Dimension and fact outputs are exported as separate files.
- Unexpected nulls stop the pipeline before loading.
- Database credentials are stored outside the code.
- Dimension-first loading supports referential integrity.
- Transactions are committed explicitly.
- Connections and cursors are closed after processing.

The ETL process therefore establishes a reproducible path from the external CSV source to the MySQL warehouse and Power BI dashboards. The resulting analytical outputs support consistent KPI definitions, outlier-sensitive comparisons, and drill-through reporting at post level.

7. Data Warehouse Design



The script creates the following database schema:

Table	Name	Primary Key	Attributes
Dimension	Time_Dim	UploadDate	Year, Month, Day
Dimension	Content_Dim	ContentID	MediaType, ContentCategory, CaptionLength, HashtagsCount
Dimension	Media_Dim	MediaID	MediaType
Dimension	Traffic_Dim	TrafficID	TrafficSource
Fact	Instagram_Fact	PostID	UploadDate, ContentID, MediaID, TrafficID, metrics, KPIs, TrafficSource

8. Power BI Dashboards

The Power BI solution converts the transformed Instagram data model into an interactive reporting environment for monitoring portfolio scale, exposure, engagement performance, time-based trends, and outlier impact. The dashboards combine portfolio-level KPIs, comparative visuals, post-level records, and outlier-sensitive analysis.

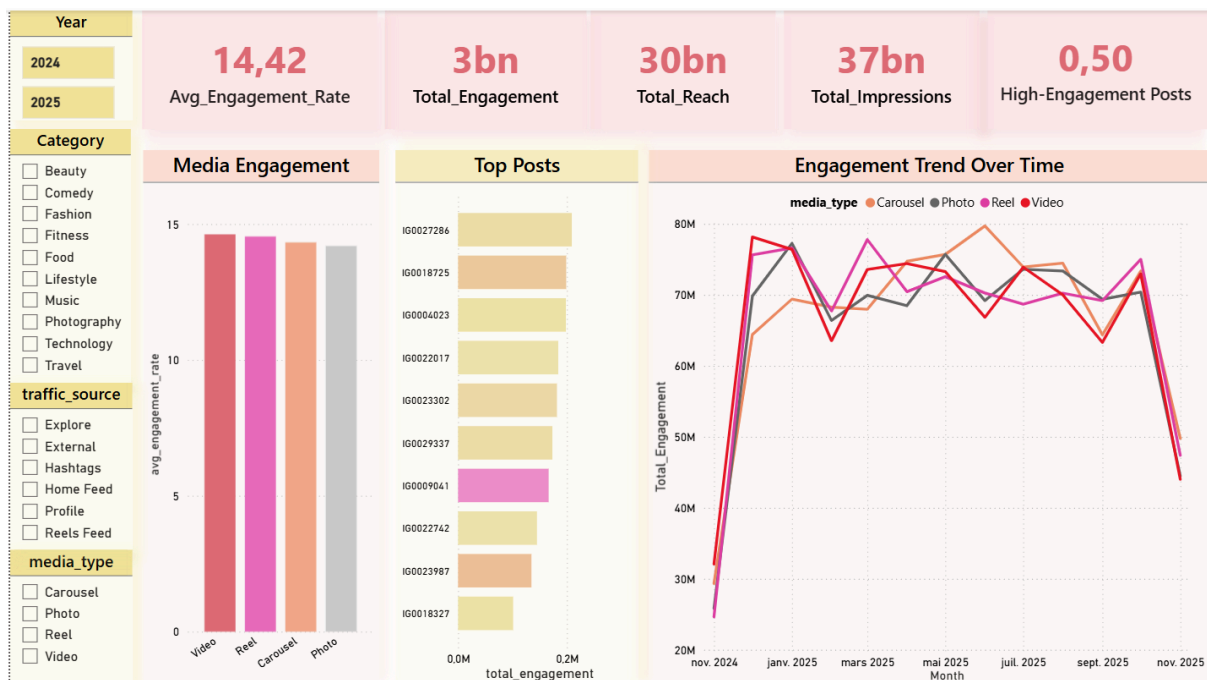
The five dashboards are designed to answer complementary analytical questions:

Dashboard	Analytical purpose
Overview	What is the overall scale and composition of portfolio performance?
Category and Channel Breakdown	How does performance vary by content category, traffic source, and time?
Performance Details	Which individual posts and records underpin aggregate results?
Outlier Impact and Distribution	How strongly do unusual engagement-rate observations affect aggregate metrics and patterns?
Further Comparison	Which category, media-type, and traffic-source differences remain after excluding outliers?

The dashboards should be interpreted together. Total engagement, reach, and impressions measure scale and exposure; engagement rate measures relative performance; time-based visuals show change; and outlier comparisons test whether apparent differences are robust.

8.1 Dashboard Overview

The Overview dashboard provides the portfolio-level context for the analysis. It combines headline KPIs, interactive slicers, media-format comparisons, top-post ranking, and monthly engagement trends.



8.1.1 KPI analysis

KPI	Value	Interpretation
Total Engagement	~3 billion	Total recorded interactions across all posts
Average Engagement Rate	14.42%	Raw arithmetic mean, including outliers
Total Reach	~30 billion	Aggregate post-level audience exposure
Total Impressions	~37 billion	Aggregate content displays, including repeated exposure
High-Engagement Posts	49.85%	Share of posts above the dataset mean for total engagement

Total engagement

Total engagement is calculated as:

$$\text{Total Engagement} = \text{Likes} + \text{Comments} + \text{Shares} + \text{Saves}$$

The portfolio generated approximately **3 billion total engagements**. This metric measures absolute interaction volume and is useful for evaluating portfolio scale, identifying high-volume posts, and comparing interaction totals across periods, formats, or segments.

However, total engagement is not adjusted for audience exposure. A post may generate substantial interaction because it reached a large audience, even if its relative engagement performance was ordinary. It should therefore be interpreted alongside reach, impressions, and engagement rate.

Average engagement rate

The dashboard reports a raw average engagement rate of **14.42%**. This value represents the arithmetic mean across all posts and includes extreme engagement-rate observations.

Because the dataset contains unusually large engagement-rate values, including observations above 50% and 300%, the raw mean is not a reliable standalone representation of typical post performance. The outlier-adjusted average of **9.95%**, presented in Section 8.4, provides a more representative estimate of typical engagement-rate performance.

Total reach

Aggregate recorded reach is approximately 30 billion. Reach represents the breadth of recorded exposure across the analyzed posts.

This value should not be interpreted as 30 billion unique individuals. The same account may be reached by multiple posts; therefore, the KPI represents cumulative post-level reach rather than a deduplicated portfolio audience.

Total impressions

Aggregate impressions are approximately **37 billion**. Impressions represent the total number of recorded displays and may include multiple exposures to the same account.

The fact that impressions exceed reach is expected because impressions capture display frequency, whereas reach represents exposure breadth. Together, these metrics provide context for interpreting total engagement and engagement rate.

High-engagement posts

Approximately **49.85% of posts** are classified as above average in total engagement. This classification is based on whether a post exceeds the dataset-wide mean total-engagement value.

The KPI is relative to the analyzed sample. It does not indicate that 49.85% of posts achieved a 49.85% engagement rate, nor does it represent an external or industry benchmark. Because the threshold uses the arithmetic mean, highly engaged posts may increase the cutoff.

8.1.2 Dashboard filters and interactivity

The Overview dashboard includes slicers for:

- **Year:** Supports comparison between 2024 and 2025 and prevents aggregate results from masking year-specific patterns.
- **Content Category:** Enables comparison across Beauty, Comedy, Fashion, Fitness, Food, Lifestyle, Music, Photography, Technology, and Travel.
- **Traffic Source:** Enables analysis across Explore, External, Hashtags, Home Feed, Profile, and Reels Feed.
- **Media Type:** Supports format-level analysis across Carousel, Photo, Reel, and Video.

These filters enable users to examine how scale, exposure, engagement, and trends change within selected segments. Results should be interpreted with awareness of the selected population size and the potential influence of outliers.

8.1.3 Media Engagement

This column chart compares average engagement volume across Video, Reel, Carousel, and Photo formats. The displayed values cluster within a relatively narrow range of approximately **85K–100K**, with Video and Reel slightly above Carousel and Photo.

The differences are modest and should not be interpreted as evidence that one format is consistently superior. This visual measures average engagement volume, not engagement rate or exposure-adjusted efficiency. In addition, averages can be influenced by high-performing observations.

The robust media-type comparison in Section 8.5 provides the more appropriate view for evaluating typical engagement-rate performance by format.

8.1.4 Top Posts by Total Engagement

The Top Posts chart ranks the ten posts with the highest total engagement. The leading posts include **IG0027286**, **IG0018725**, and **IG0004023**, with top-ten values ranging from approximately **100K to nearly 200K engagements**.

The chart indicates that interaction volume is concentrated among a relatively small number of posts. It is useful for identifying records for creative review and winner-pattern analysis.

However, total-engagement ranking does not measure efficiency. A top-ranked post may have achieved high engagement because it received substantial exposure. These posts should therefore be reviewed alongside reach, impressions, engagement rate, follower growth, media type, traffic source, and publishing characteristics.

8.1.5 Engagement Trend Over Time

The line chart shows monthly total engagement by media type. Performance varies materially across time and format:

- Carousel shows notable peaks around June and November.
- Reel peaks around March and increases again toward December.
- Video declines sharply in February and September before recovering later in the period.
- Photo is comparatively more stable but still exhibits variation.

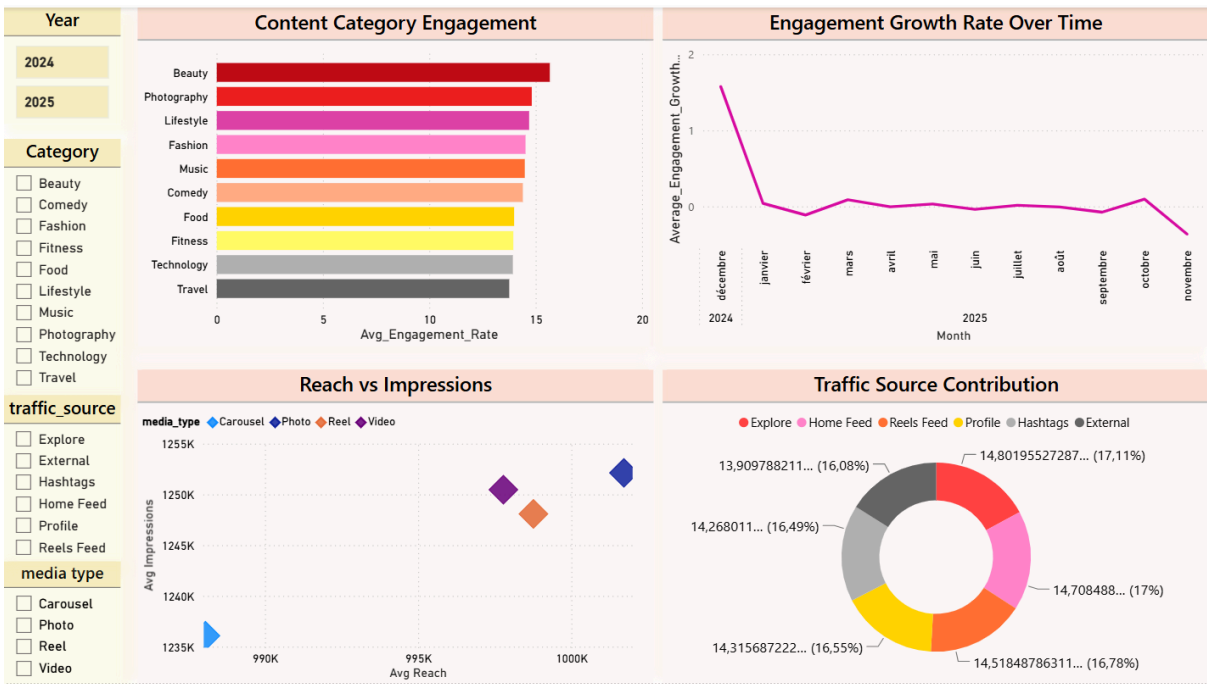
The chart confirms that engagement is not constant throughout the reporting period. These movements may reflect changes in post volume, content mix, campaign activity, media distribution, audience behaviour, or exceptional individual posts.

The visual identifies when performance changed; it does not explain why. Any operational interpretation should be validated using monthly post counts, category

mix, media mix, traffic-source distribution, campaign information, and top-post concentration.

8.2 Category & Channel Breakdown

This dashboard examines how engagement and exposure are distributed across content categories, traffic sources, and reporting periods. It combines category-level averages, traffic-source contribution, monthly engagement momentum, and exposure relationships.



8.2.1 Content Category Engagement

In the raw dataset, Beauty records the highest average engagement rate at **15.66%**, followed by Photography (**14.82%**) and Lifestyle (**14.69%**). Travel (**13.76%**) and Technology (**13.93%**) record the lowest raw averages.

Although this produces a category ranking, the values are concentrated within a relatively narrow range. The raw difference between Beauty and Travel is **1.91 percentage points**.

This ranking should be treated as an initial descriptive result rather than evidence that Beauty consistently outperforms other categories. The outlier-adjusted analysis in Sections 8.4 and 8.5 shows that category differences narrow substantially after extreme engagement-rate observations are excluded.

8.2.2 Traffic Source Contribution

The donut chart shows the contribution of six traffic sources to total measured engagement:

Traffic source	Approximate contribution
Explore	17.11%
Home Feed	Approximately 16-17%
Reels Feed	Approximately 16-17%
Profile	Approximately 16-17%
Hashtags	Approximately 16-17%
External	16.08%

The distribution is broadly balanced. No single traffic source dominates total engagement within the observed dataset.

This chart measures contribution, not efficiency. A traffic source may account for a larger engagement share because it has more posts, greater reach, or higher exposure volume. Its effectiveness should also be assessed using exposure-adjusted measures, follower acquisition, campaign objective, and conversion data where available.

8.2.3 Engagement Growth Rate Over Time

This visual shows the month-over-month percentage change in total engagement from December 2024 to November 2025.

Period	Growth rate	Interpretation
December 2024	+157.6%	Strongest positive monthly increase
January 2025	+4.1%	Growth normalizes after December
February 2025	-11.3%	Meaningful monthly decline
March 2025	+8.8%	Moderate recovery
September 2025	-7.6%	Notable decline
October 2025	+9.6%	Short-term rebound
November 2025	-36.4%	Largest monthly decline

Most monthly changes during 2025 remain close to zero, indicating limited sustained month-over-month momentum. The largest movements occur at the beginning and end of the reporting period.

Growth rate measures change relative to the previous month, not the absolute level of engagement. A large change may result from an unusually low or high comparison month. The chart should therefore be interpreted with monthly engagement totals, post volume, content mix, media mix, traffic-source distribution, and exceptional-post concentration.

The observed period is insufficient to establish recurring seasonality.

8.2.4 Reach versus Impressions

The scatter plot shows a positive descriptive association between average reach and average impressions. Posts or groups with greater recorded reach generally also accumulate more total impressions.

This is consistent with the relationship between exposure breadth and display frequency. Content shown to more accounts has greater opportunity to generate total displays, including repeat exposures.

The visual demonstrates association only. It does not establish that higher reach independently causes higher impressions, as both may be influenced by content distribution, account characteristics, post volume, campaign activity, and platform behaviour.

8.3 Performance Details

All Media Selected Media Type			All Categories Selected Content Category			Hashtags Selected Traffic Source				
Advanced Analytics & Drill-Through Insights										
post_id	media_type	content_category	total_engagement	engagement_rate	followers_gained	Année	Trimestre	Mois	Jour	traffic_source
IG0000002	Photo	Fitness	75185	5,59	805	2025	Qtr 3	août	15	Hashtags
IG0000010	Photo	Music	88019	5,65	611	2025	Qtr 1	mars	10	Hashtags
IG0000012	Reel	Beauty	88822	5,70	240	2025	Qtr 2	mai	3	Hashtags
IG0000016	Reel	Music	191531	8,66	752	2024	Qtr 4	décembre	6	Hashtags
IG0000026	Video	Fashion	87027	333,54	243	2025	Qtr 1	février	18	Hashtags
IG0000040	Reel	Fitness	31774	1,83	907	2025	Qtr 3	juillet	10	Hashtags
IG0000047	Reel	Travel	67914	6,63	49	2025	Qtr 2	mai	17	Hashtags
IG0000052	Photo	Fashion	84297	5,47	870	2025	Qtr 4	octobre	15	Hashtags
IG0000061	Carousel	Fitness	148056	18,39	80	2024	Qtr 4	décembre	20	Hashtags
IG0000066	Photo	Photography	172971	53,46	886	2025	Qtr 3	juillet	6	Hashtags
IG0000075	Video	Beauty	108092	5,19	215	2025	Qtr 1	janvier	23	Hashtags
IG0000080	Carousel	Comedy	88118	3,94	550	2025	Qtr 2	juin	2	Hashtags
IG0000082	Photo	Comedy	83466	4,47	52	2025	Qtr 2	avril	11	Hashtags
IG0000098	Video	Technology	195539	33,55	914	2024	Qtr 4	décembre	18	Hashtags
IG0000099	Photo	Comedy	131657	9,87	340	2025	Qtr 1	janvier	31	Hashtags
IG0000108	Carousel	Beauty	201381	14,90	442	2025	Qtr 2	juin	2	Hashtags
IG0000112	Photo	Food	143884	16,38	109	2025	Qtr 3	août	15	Hashtags
IG0000122	Photo	Photography	26697	12,95	291	2024	Qtr 4	décembre	22	Hashtags
IG0000130	Video	Technology	90176	37,91	622	2025	Qtr 3	août	14	Hashtags
IG0000140	Reel	Fitness	203189	12,03	337	2025	Qtr 3	août	21	Hashtags
IG0000141	Photo	Photography	102402	5,79	622	2025	Qtr 1	janvier	20	Hashtags

The Performance Details dashboard provides post-level visibility into the records underlying aggregate KPIs and charts. It connects portfolio-level results with individual observations.

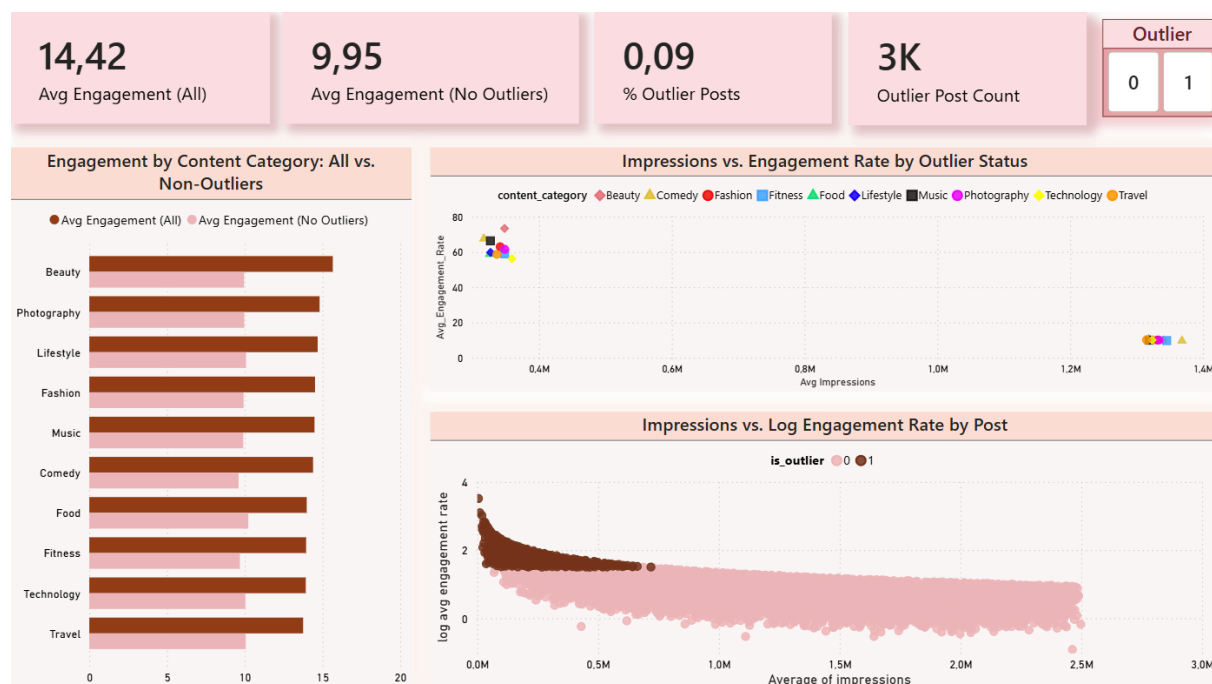
The dashboard supports:

- Identification of individual posts and their recorded performance.
- Review of total engagement, engagement rate, reach, impressions, and followers gained.
- Comparison of content category, media type, and traffic source.
- Investigation of high-performing and unusual posts.
- Drill-through from aggregated dashboard visuals to post-level records.

This page is particularly useful for validating whether a trend is broad-based or dominated by a small number of posts. For example, a strong category average may be supported by consistent performance across many records or disproportionately influenced by a limited number of exceptional posts.

The table provides evidence for further investigation. It should not be interpreted as proof that any individual post attribute caused the observed result.

8.4 Outlier Impact and Distribution



The Outlier Impact and Distribution dashboard evaluates the effect of unusual engagement-rate observations on aggregate metrics, category comparisons, and the relationship between impressions and engagement rate.

The dashboard uses the `is_outlier` flag created during ETL using the Interquartile Range method. Outlier posts are retained in the data model rather than removed, enabling transparent comparison between all observations, non-outliers, and outliers.

8.4.1 DAX KPI measures

Average engagement rate_all posts

```
Avg Engagement (All) =  
AVERAGE(instagram_fact[engagement_rate])
```

The all-post average engagement rate is **14.42%**. It includes every post in the dataset, including statistically unusual engagement-rate observations.

This KPI serves as the baseline for measuring the influence of outliers.

Average engagement rate_no outliers

```
Avg Engagement (No Outliers) =  
CALCULATE(AVERAGE(instagram_fact[engagement_rate]),  
instagram_fact[is_outlier] = 0)
```

After excluding posts flagged as outliers, average engagement rate decreases to **9.95%**.

$$14.42\% - 9.95\% = 4.47 \text{ percentage points}$$

This represents an approximate **31% reduction** relative to the raw all-post average.

The comparison demonstrates that extreme observations materially inflate the raw arithmetic mean. The all-post KPI should be interpreted as a full-population measure, whereas the non-outlier KPI is more suitable for describing typical post performance.

Outlier post count

```
Outlier Post Count = CALCULATE(COUNTROWS(instagram_fact),  
instagram_fact[is_outlier] = 1)
```

The dashboard displays approximately **3K outlier posts**, representing **2,557 observations** before rounding.

Although these posts represent only approximately **8.5%** of the dataset, their removal changes the average engagement rate materially. Their influence is therefore disproportionate to their population share.

Outlier proportion

% Outlier Posts =

```
DIVIDE(  
  
    CALCULATE(COUNTROWS(instagram_fact),  
instagram_fact[is_outlier] = 1),  
  
    COUNTROWS(Instagram_Fact))
```

Approximately **8.5% of posts** are classified as engagement-rate outliers.

Log-transformed engagement rate

log avg engagement rate =

```
LOG(content_dim[Avg_Engagement_Rate])
```

The dashboard uses a logarithmic transformation of engagement rate to reduce the visual influence of extreme values. This allows the central distribution, long-tail behaviour, and separation between outlier and non-outlier observations to be interpreted more clearly.

The log transformation changes the visual scale only. It does not remove records or alter the underlying outlier classification.

8.4.2 Outlier Slicer

The outlier slicer enables direct filtering by outlier classification.

Value	Classification
0	Non-outlier
1	Engagement-rate outlier

Including `is_outlier` in the fact table integrates anomaly detection into the reporting model. Users can compare all posts, non-outliers, and outliers using the same dimensions, measures, and visuals.

This design is preferable to maintaining separate datasets because it keeps the analysis reproducible and preserves the complete post population.

8.4.3 Engagement by Content Category: All vs. Non-Outliers

This clustered bar chart compares category-level average engagement rate before and after excluding engagement-rate outliers.

Measure	Raw result	Non-outlier result
Highest category	Beauty: 15.66%	Food: 10.23%
Lowest category	Travel: 13.76%	Comedy: 9.61%
Range	1.90 percentage points	0.62 percentage points

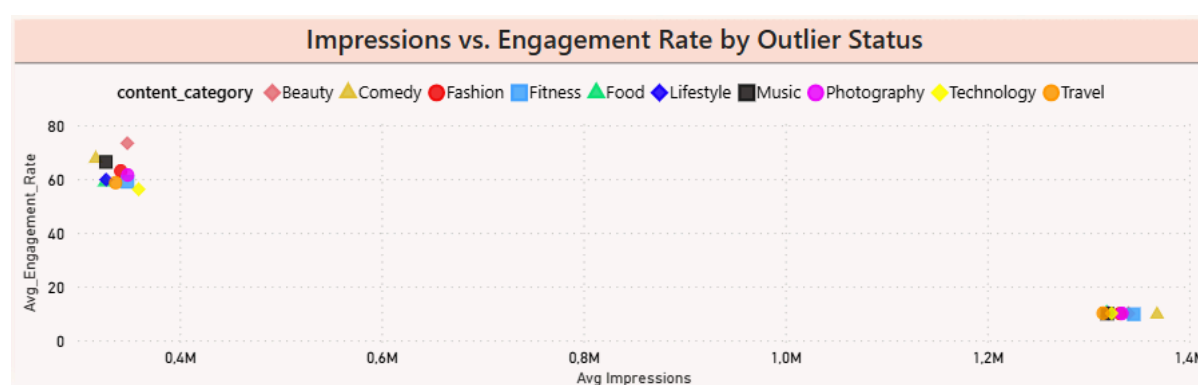
In the full dataset, Beauty appears to be the leading category. After outlier exclusion, Food has the highest average engagement rate, while Beauty moves to approximately sixth place.

Beauty declines from **15.66% to 9.96%**, a reduction of **5.71 percentage points**. Food declines from **13.99% to 10.23%**, a smaller reduction of **3.76 percentage points**. Travel also experiences a comparatively small decline of **3.69 percentage points**.

Outlier counts are similar across categories, ranging from approximately **226 for Travel** to **286 for Photography**. Therefore, the ranking changes are not caused by one category containing nearly all outliers.

Interpretation: The raw category ranking is not robust. After outlier exclusion, typical category engagement rates cluster around 10%, indicating that category differences are substantially smaller than the raw results suggest.

8.4.4 Impressions vs. Engagement Rate by Outlier Status



This scatter plot compares impressions and engagement rate by outlier status, with content category represented by colour.

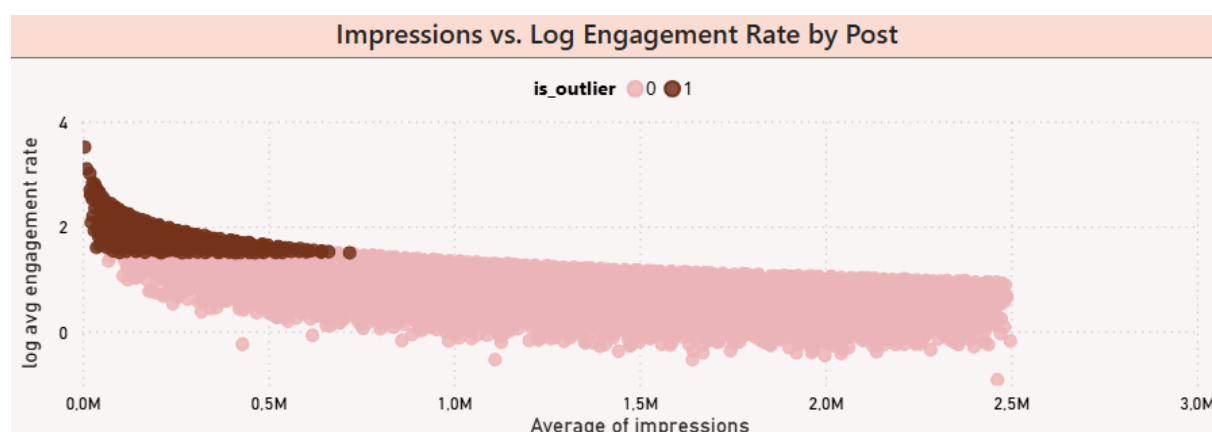
Two distinct clusters are visible:

Population	Approximate impressions	Approximate engagement rate
Outliers	0.3-0.35M	55-75%
Non-outliers	1.3-1.4M	9-12%

The separation indicates that outliers form a distinct analytical population rather than minor deviations from the main distribution. Outliers are concentrated in a lower-impression, high-engagement-rate region, while non-outliers occupy a higher-impression, more moderate-rate region.

This supports the decision to analyze outliers separately. However, it does not demonstrate that lower impressions cause higher engagement rates. The relationship may reflect denominator effects, post type, account context, campaign conditions, or measurement design.

8.4.5 Impressions vs. Log Engagement Rate by Post

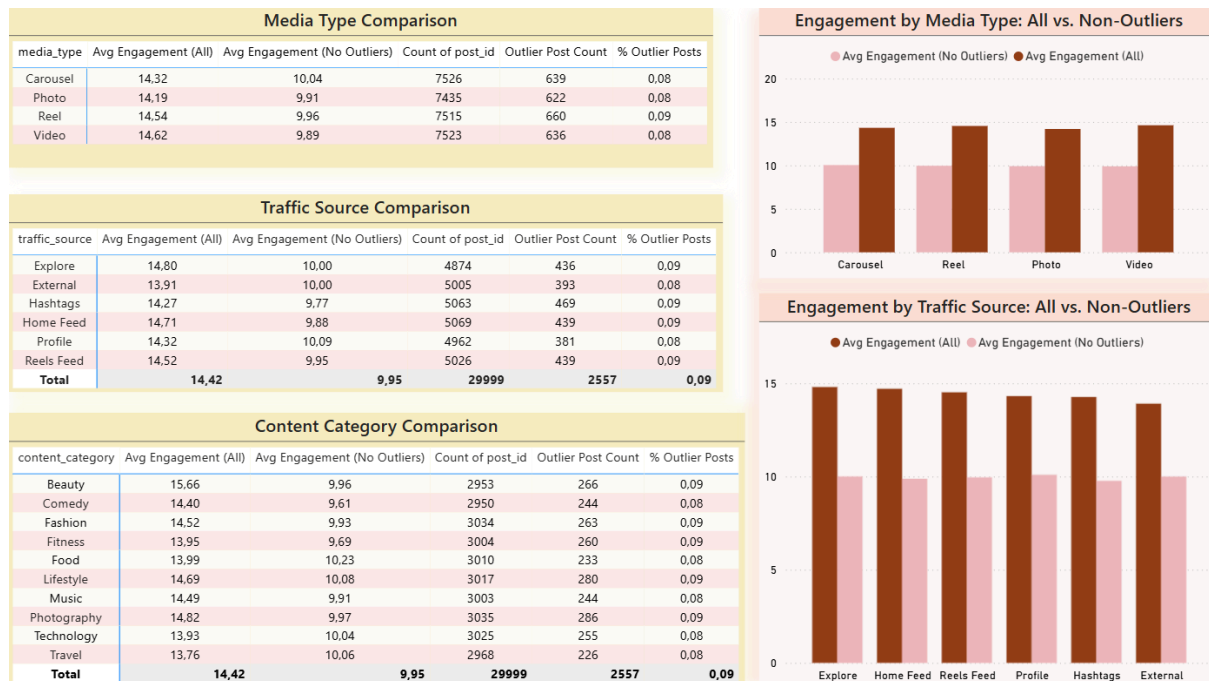


This post-level scatter plot uses log-transformed engagement rate to improve visibility of the underlying distribution.

Non-outlier posts extend across a broad impression range, reaching approximately **2.5 million impressions**, with engagement rates concentrated at lower log-scaled values. Outlier posts are concentrated at lower impression levels and materially higher log engagement-rate values.

The visual provides graphical support for the IQR-based outlier classification. The log scale reduces distortion from large engagement-rate values but does not imply a causal relationship between impressions and engagement rate.

8.5 Further Comparison



The Further Comparison dashboard evaluates whether apparent differences in engagement-rate performance remain after excluding potentially influential observations.

8.5.1 Engagement by Media Type: All vs. Non-Outliers

Media Type	All posts	Non-outliers
Video	14.62%	9.89%
Reel	14.54%	9.96%
Carousel	14.32%	10.04%
Photo	14.19%	9.91%

The raw media-type range is **0.43 percentage points**, narrowing to **0.15 percentage points** after outlier exclusion.

Video changes from the highest raw average to the lowest non-outlier average, while Carousel records the highest non-outlier average. This confirms that the raw format ranking is sensitive to extreme engagement-rate observations.

Interpretation: Typical engagement-rate performance is highly similar across formats. Media decisions should consider campaign objective, content suitability,

production cost, reach, impressions, follower growth, and conversion outcomes rather than small raw engagement-rate differences.

8.5.2 Engagement by Traffic Source: All vs. Non-Outliers

Raw traffic-source averages are generally between **14% and 15%**. After excluding outliers, the values converge around **10%**.

Measure	All posts	Non-outlier posts
Highest traffic source	Explore: 14.80%	Profile: 10.09%
Lowest traffic source	External: 13.91%	Hashtags: 9.77%
Range	0.89 percentage points	0.32 percentage points

The apparent raw advantage of Explore does not persist after outlier exclusion.

Interpretation: Traffic source has a limited standalone relationship with typical engagement-rate performance. The result should not be interpreted as evidence that all traffic sources are equally effective for awareness, follower growth, conversion, or cost efficiency.

8.5.3 Content Category Comparison

Content Category Comparison					
content_category	Avg Engagement (All)	Avg Engagement (No Outliers)	Count of post_id	Outlier Post Count	% Outlier Posts
Beauty	15,66	9,96	2953	266	0,09
Comedy	14,40	9,61	2950	244	0,08
Fashion	14,52	9,93	3034	263	0,09
Fitness	13,95	9,69	3004	260	0,09
Food	13,99	10,23	3010	233	0,08
Lifestyle	14,69	10,08	3017	280	0,09
Music	14,49	9,91	3003	244	0,08
Photography	14,82	9,97	3035	286	0,09
Technology	13,93	10,04	3025	255	0,08
Travel	13,76	10,06	2968	226	0,08
Total	14,42	9,95	29999	2557	0,09

The category comparison table presents:

- Average engagement rate for all posts.
- Average engagement rate excluding outliers.
- Post count.
- Outlier count.
- Outlier proportion.

Category sample sizes are balanced, ranging from approximately **2,950 to 3,035 posts**. Outlier rates are also broadly consistent at approximately **8–9%**.

The table provides the numerical evidence behind the category comparison visual: Beauty leads in the raw data, Food leads after outlier exclusion, and category differences narrow materially in the robust comparison.

Interpretation: Outlier-adjusted averages provide a more reliable basis for comparing typical category engagement than raw means alone.

8.5.4 Traffic Source Comparison

Traffic Source Comparison					
traffic_source	Avg Engagement (All)	Avg Engagement (No Outliers)	Count of post_id	Outlier Post Count	% Outlier Posts
Explore	14,80	10,00	4874	436	0,09
External	13,91	10,00	5005	393	0,08
Hashtags	14,27	9,77	5063	469	0,09
Home Feed	14,71	9,88	5069	439	0,09
Profile	14,32	10,09	4962	381	0,08
Reels Feed	14,52	9,95	5026	439	0,09
Total	14,42	9,95	29999	2557	0,09

In the raw dataset, Explore records the highest average engagement rate at **14.80%**, followed by Home Feed (**14.71%**) and Reels Feed (**14.52%**). External records the lowest average at **13.91%**.

After outlier exclusion, Profile records the highest average at **10.09%**, while Hashtags records the lowest at **9.77%**.

The range narrows from **0.89** to **0.32 percentage points**.

Interpretation: Traffic-source rankings are not stable under outlier exclusion. The difference between sources is modest in the non-outlier population, indicating that raw traffic-source rankings should not be used alone to determine channel strategy.

8.5.5 Media Type Comparison

Media Type Comparison					
media_type	Avg Engagement (All)	Avg Engagement (No Outliers)	Count of post_id	Outlier Post Count	% Outlier Posts
Carousel	14,32	10,04	7526	639	0,08
Photo	14,19	9,91	7435	622	0,08
Reel	14,54	9,96	7515	660	0,09
Video	14,62	9,89	7523	636	0,08

The media comparison table confirms that outlier rates are similar across Carousel, Photo, Reel, and Video, at approximately **8–9%**. This indicates that unusual engagement-rate observations are distributed across formats rather than concentrated within one media type.

After outlier exclusion, all media-type averages remain close to 10%, with only a **0.15 percentage-point** difference between the highest and lowest formats.

Interpretation: The robust results support a diversified media strategy. No format demonstrates a sufficiently large or stable engagement-rate advantage to justify major portfolio reallocation based on this metric alone.

9. Key Findings

9.1 Portfolio scale and performance

The analyzed portfolio contains **29,999 posts** and generated approximately:

- **3 billion total engagements**
- **30 billion aggregate reach**
- **37 billion aggregate impressions**
- **14.42% raw average engagement rate**
- **49.85% of posts above the dataset mean for total engagement**

Total engagement measures the volume of recorded interactions—likes, comments, shares, and saves—while reach and impressions describe exposure breadth and frequency. These metrics demonstrate substantial portfolio scale; however, they represent aggregated post-level records and should not be interpreted as unique users or unique views across the full reporting period.

Interpretation: High aggregate scale does not imply consistent performance across individual posts. Portfolio-level metrics should therefore be interpreted alongside post-level distributions, top-post analysis, exposure measures, and robust engagement-rate indicators.

9.2 Engagement is concentrated

The Top Posts visual shows that a relatively small number of posts generate substantially higher total engagement than the remaining posts in the top-ten group. Engagement is therefore unevenly distributed across the portfolio, with exceptional posts contributing disproportionately to total interaction volume.

Interpretation: The approximately 3 billion total-engagement result should not be interpreted as evidence that all posts perform at a similar level. Post-level analysis is

required to determine whether exceptional performance reflects repeatable content characteristics, unusual exposure, campaign effects, or isolated events.

9.3 Engagement growth is volatile

Monthly total engagement varies across media types, with visible peaks and declines throughout the reporting period. Month-over-month growth is particularly variable at the beginning and end of the observed series:

Period	Engagement growth rate
December 2024	+157.6%
February 2025	-11.3%
March 2025	+8.8%
September 2025	-7.6%
October 2025	+9.6%
November 2025	-36.4%

Most months during 2025 record relatively small changes around zero. This indicates limited sustained engagement momentum rather than a persistent upward or downward trajectory.

Interpretation: The December 2024 increase and November 2025 decline are notable movements, but they should not be treated as evidence of recurring seasonality. Additional analysis of posting volume, media mix, content categories, campaign activity, reach, impressions, and top-post concentration is required before assigning operational or seasonal explanations.

9.4 Reach and impressions are positively associated

The reach-versus-impressions scatter plot shows a positive descriptive relationship: posts or groups with higher reach generally also record higher impressions. This is consistent with broader audience distribution generating greater opportunities for repeated content exposure.

Interpretation: Reach and impressions should be evaluated together. Reach reflects exposure breadth, while impressions measure total display volume and may include repeated exposure to the same accounts. The visual demonstrates association only and does not establish that increasing reach independently causes impressions to increase.

9.5 Category performance is not ranking-stable

Content-category engagement-rate rankings change substantially after excluding engagement-rate outliers.

Measure	Raw result	Non-outlier result
Highest category	Beauty: 15.66%	Food: 10.23%
Lowest category	Travel: 13.76%	Comedy: 9.61%
Range	1.90 percentage points	0.62 percentage points

In the raw data, Beauty appears to be the strongest category. After excluding outliers, however, Food has the highest average engagement rate, while Beauty moves to approximately sixth place.

Beauty shows the largest reduction, declining from **15.66% to 9.96%**, or **5.71 percentage points**. Food declines from **13.99% to 10.23%**, a smaller reduction of **3.76 percentage points**. Travel also shows a comparatively small reduction, falling from **13.76% to 10.06%**.

Outlier counts are broadly distributed across all ten categories, ranging from **226 posts for Travel to 286 posts for Photography**. Therefore, the category-ranking changes are not explained by a single category containing most outlier observations.

Interpretation: The apparent category differences in raw engagement rate are not robust. After removing statistically unusual engagement-rate observations, category performance converges around 10%, indicating that no category demonstrates a large or stable standalone advantage in typical engagement-rate performance.

9.6 Outlier materially influence the average engagement rate

A total of **2,557 posts**, representing approximately **8.5%** of the dataset, are classified as engagement-rate outliers under the IQR rule.

Removing these observations reduces the average engagement rate from **14.42%** to **9.95%**: **14.42% – 9.95% = 4.47 percentage points**

This represents an approximate **31% reduction** relative to the raw average.

Outlier observations form a visually distinct group, generally characterized by lower average impressions—approximately **0.3–0.35 million**—and exceptionally high

engagement rates of approximately **55–75%**. Their impact on the overall mean is therefore disproportionate to their population share.

Interpretation: The non-outlier average of 9.95% is more suitable for describing typical post performance. The raw average of 14.42% should be retained as a full-population metric but interpreted with clear acknowledgement of outlier influence.

9.7 Media-type differences are limited

Raw average engagement rates vary only modestly across formats

Media type	All posts	Non-outlier posts
Video	14.62%	9.89%
Reel	14.54%	9.96%
Carousel	14.32%	10.04%
Photo	14.19%	9.91%

The range narrows from **0.43 percentage points** in the raw data to **0.15 percentage points** after outlier exclusion. Video changes from the highest raw average to the lowest non-outlier average, while Carousel becomes the highest non-outlier format.

Interpretation: Media formats perform similarly in typical engagement-rate terms. Minor raw-average differences are not sufficient evidence to prioritize one format over another without considering reach, impressions, production cost, follower acquisition, campaign objectives, and conversion performance.

9.8 Traffic-source differences are limited

Traffic-source averages also converge after outlier removal:

Measure	All posts	Non-outlier posts
Highest traffic source	Explore: 14.80%	Profile: 10.09%
Lowest traffic source	External: 13.91%	Hashtags: 9.77%
Range	0.89 percentage points	0.32 percentage points

Explore appears strongest in the raw data, but its advantage does not remain distinct after outlier exclusion. In the non-outlier comparison, all traffic sources cluster near 10%.

The traffic-source contribution visual also shows a balanced distribution, with each source contributing approximately **16–17%** of total engagement.

Interpretation: Traffic-source rankings are sensitive to extreme engagement-rate observations. Traffic source alone has a limited association with typical engagement-rate performance and should be considered alongside reach, impressions, follower gain, and broader campaign context.

9.9 Category and traffic source explain standalone variation

The exploratory correlation analysis found negligible linear associations between engagement rate and the one-hot-encoded content-category and traffic-source variables.

- The largest positive coefficient is associated with **Beauty**: $r=0.0139$
- The largest negative coefficient is associated with **External traffic**: $r=-0.0077$
- All coefficients fall below $|r|=0.014$

These results indicate that content category and traffic source individually explain very little of the observed variation in engagement rate.

This finding is consistent with the outlier-adjusted dashboard comparisons, where category and traffic-source averages converge substantially after extreme observations are excluded.

Interpretation: Category and traffic source should not be treated as strong standalone drivers of engagement rate. However, negligible pairwise correlations do not prove that these variables have no strategic relevance. The analysis does not account for interactions with media type, exposure, posting time, account characteristics, paid activity, content quality, audience composition, or campaign objectives.

9.10 Total engagement and engagement rate answer different questions

Total engagement measures absolute interaction volume. Engagement rate measures relative engagement performance according to the source-defined exposure basis.

A category, format, or traffic source may generate high total engagement because it receives greater exposure or contains more posts, without having the highest engagement rate. Conversely, a post may report a high engagement rate while generating limited total engagement because its exposure level is relatively low.

Interpretation: The analysis should not identify a single “best” category, media type, or traffic source without first defining the business objective. Total engagement is appropriate for assessing interaction volume, while engagement rate is more appropriate for assessing relative engagement performance.

10. Business Recommendations

10.1 Use raw and robust KPIs together

Use the **non-outlier average engagement rate of 9.95%** as the primary benchmark for typical post performance. Continue reporting the **14.42% all-post average**, but clearly label it as a raw measure that includes extreme engagement-rate observations.

Future dashboard versions should add:

- Median engagement rate.
- Interquartile range.
- Selected percentile measures.
- Trimmed mean.
- Distribution plots.
- Confidence intervals where appropriate.

The outlier slicer should remain available so users can compare:

- All posts.
- Non-outlier posts.
- Outlier posts.

This approach distinguishes repeatable typical performance from exceptional historical outcomes.

10.2 Use rankings to prioritize testing, not declare winners

The category, media-type, and traffic-source rankings should not be used alone to allocate budget or define strategy. The non-outlier results show that typical engagement-rate differences are small:

- Category range: **0.62 percentage points**.
- Media-type range: **0.15 percentage points**.
- Traffic-source range: **0.32 percentage points**.

Food, Lifestyle, Travel, and Technology appear at the upper end of the non-outlier category distribution, but their differences from other categories are limited. These categories should be treated as testing priorities rather than confirmed strategic winners.

Future tests should compare similar content concepts while considering:

- Media format.
- Reach and impressions.
- Posting date and time.
- Traffic source.
- Caption and hashtag characteristics.
- Posting frequency.
- Campaign activity.
- Follower-growth outcomes.
- Production cost.
- Business objective.

A structured testing approach is more reliable than increasing content volume solely because a category has the highest historical average.

10.3 Analyze exceptional posts separately

Develop a recurring winner-pattern analysis for top-engagement posts and engagement-rate outliers. For each selected post, review:

- Media type.
- Content category.
- Traffic source.
- Reach and impressions.
- Followers gained.
- Caption length.
- Hashtag count.
- Publication date and time.
- Campaign or promotional status, where available.

High engagement-rate outliers should be manually reviewed to determine whether they may reflect:

- Exceptional or viral content.
- Low exposure denominators.
- Paid promotion.
- Influencer activity.
- Campaign effects.
- Measurement issues.

- Data-quality anomalies.

The purpose is to identify potentially repeatable characteristics, not to infer that an observed attribute caused the post's performance.

Track the outlier proportion over time. An unexpected increase may indicate changes in content strategy, campaign activity, platform distribution, or data-collection methodology.

10.4 Maintain media and traffic-source diversity

The robust media-type averages cluster tightly around 10%. The evidence does not support a substantial shift toward a single format based solely on engagement rate.

Maintain a diversified media strategy and evaluate formats using a broader decision framework:

- Total engagement.
- Reach.
- Impressions.
- Followers gained.
- Engagement rate.
- Production cost.
- Campaign objective.
- Conversion performance.

Similarly, maintain a diversified traffic-source strategy. The balanced traffic-source contribution and narrow non-outlier engagement-rate range do not justify heavy reallocation toward one source.

10.5 Investigate temporal anomalies

The December 2024 increase of approximately **157.6%** and November 2025 decline of approximately **36.4%** should be investigated through a monthly diagnostic process.

For each unusual period, compare:

- Number of posts published.
- Media-type mix.
- Content-category mix.
- Traffic-source distribution.
- Average reach and impressions.
- Top-post concentration.

- Campaign or external-event indicators.
- Paid versus organic activity, where available.
- Outlier proportion.

This process can help distinguish volume effects, campaign effects, partial-period effects, exposure changes, and exceptional posts from genuine performance changes.

10.6 Separate scale and efficiency reporting

Maintain two complementary reporting views.

Reporting view	Primary measures	Business purpose
Scale	Total engagement, reach, impressions, top posts, monthly interaction volume	Monitor portfolio visibility and interaction volume
Efficiency	Raw engagement rate, non-outlier engagement rate, median engagement rate, engagement by exposure level	Monitor typical relative performance

This structure prevents stakeholders from interpreting high interaction volume as evidence of high engagement efficiency.

10.7 Strengthen dashboard governance

Future dashboard versions should:

- Display outlier proportion explicitly as **8.5%**, rather than **0.09**.
- Document the engagement-rate formula and denominator.
- Distinguish clearly between total engagement and engagement rate.
- Add median, percentile, and trimmed-mean measures.
- Display post counts beside category, media-type, and traffic-source averages.
- Add tooltips stating that visuals describe associations rather than causality.
- Add confidence intervals or uncertainty measures where appropriate.
- Maintain post-level drill-through for top posts and outliers.
- Monitor monthly post volume alongside monthly engagement and growth rate.

11. Analytical Limitations

11.1 Engagement-rate definition

The source dataset does not explicitly document the engagement-rate formula or denominator. The rate may be based on reach, impressions, followers, or another exposure measure.

Without this definition, the analysis can compare the supplied engagement-rate values but cannot make definitive claims about audience-level engagement efficiency.

11.2 Outlier classification

The IQR method identifies statistically unusual observations; it does not prove that an observation is incorrect, irrelevant, or commercially unimportant.

Some outliers may represent genuinely successful posts. They should therefore be retained and analyzed as a separate population rather than deleted automatically.

The selected $1.5 \times \text{IQR}$ threshold affects classification. Alternative approaches—such as percentile trimming, robust z-scores, or segment-specific thresholds—may identify different observations. A single global threshold may also overlook differences in normal engagement-rate behaviour across categories, formats, audience sizes, and time periods.

The category-level results are also dependent on this rule. The convergence of category averages after exclusion should be interpreted as evidence that raw averages are sensitive to extreme values, not as proof that content category has no relationship with engagement.

11.3 Descriptive analysis and omitted variables

The project is descriptive and observational. It does not establish causal relationships.

The correlation analysis measures unadjusted pairwise linear association between engagement rate and binary category or traffic-source indicators. It does not account for nonlinear relationships, interaction effects, or confounding variables.

The dataset does not include several factors that may influence post performance, including:

- Paid promotion.
- Campaign objective.
- Advertising budget.
- Account size.
- Audience demographics.
- Content quality.

- Publication strategy.
- Influencer activity.
- Competitor activity.
- External events.
- Conversion outcomes.

Accordingly, the findings should not be interpreted as evidence that a category, format, traffic source, or exposure measure causes better engagement.

11.4 Exposure and aggregation

Aggregate reach should not be interpreted as the number of unique users reached across the full portfolio, because the same account may be exposed to multiple posts.

Impressions include repeated exposure and therefore measure display frequency rather than a distinct audience.

High total engagement may result from high exposure rather than high relative engagement efficiency. Volume measures should always be interpreted alongside reach, impressions, and engagement rate.

11.5 Time coverage

The growth-rate analysis covers December 2024 to November 2025. This period is insufficient to establish reliable recurring seasonality.

Monthly growth rates are also sensitive to the prior month's value. A small or unusually large previous month can produce an exaggerated percentage change.

11.6 Data source and representation

The Kaggle dataset is an externally sourced analytical sample rather than a validated production export. The findings apply only to the supplied observations.

Future work should validate:

- Record completeness.
- Sampling consistency.
- Traffic-source attribution.
- Metric definitions.
- The number and type of accounts represented.
- Whether the dataset reflects a real, synthetic, or aggregated portfolio.

11.7 Mean-based metric sensitivity

Means remain sensitive to distribution shape even after outlier classification. The dashboard should therefore complement averages with median values, percentiles, trimmed means, and distributional visuals.

11.8 Relative high-engagement classification

The `high_engagement_flag` identifies posts whose total engagement exceeds the dataset mean. It is a relative internal classification, not an external performance benchmark.

Future versions should consider thresholds based on:

- Historical percentiles.
- Category-specific benchmarks.
- Media-type-specific benchmarks.
- Account-level benchmarks.
- Business-defined targets.

11.9 Limited business outcomes

The project focuses on engagement and exposure. It does not directly measure:

- Sales.
- Leads.
- Website visits.
- Revenue.
- Conversion rate.
- Return on advertising spend.
- Content-production cost.

Engagement should therefore be treated as an intermediate performance indicator rather than a complete measure of business value.

12. Conclusion

This project provides an integrated Business Intelligence solution for monitoring Instagram performance across interaction volume, audience exposure, engagement rate, content categories, media formats, traffic sources, and time periods.

The portfolio demonstrates substantial aggregate scale, generating approximately **3 billion total engagements**, **30 billion aggregate reach**, and **37 billion aggregate**

impressions across **29,999 posts**. However, performance is heterogeneous, with a relatively small number of posts contributing disproportionately to total engagement.

The most important analytical finding is the influence of engagement-rate outliers. Excluding **2,557 posts**, representing approximately **8.5%** of the dataset, reduces the average engagement rate from **14.42% to 9.95%**—a reduction of approximately **31%**. This indicates that the raw average materially overstates typical post performance.

After outlier exclusion, category, media-type, and traffic-source averages converge substantially. The findings show that raw rankings are not stable enough to support strong strategic conclusions about a single best category, format, or traffic source.

The recommended reporting approach is to distinguish between:

- **Scale:** Total engagement, reach, impressions, and top-post performance.
- **Efficiency:** Non-outlier engagement rate, median engagement rate, and performance relative to exposure.

The dashboards should be used to monitor performance, identify unusual observations, support post-level investigation, and prioritize controlled testing. Future analysis should incorporate campaign, cost, audience, publishing-time, and conversion data to determine which content strategies generate sustainable business value rather than engagement alone.

